

MEG-002 — Event-Driven Runtime

External publication draft

MEG-002 — Event-Driven Runtime

Software should not ask what happened. It should be told.

Purpose

The Mosaic Runtime is built upon an event-driven architecture.

Rather than components communicating directly through tightly coupled service calls, the runtime is composed of autonomous capabilities that communicate through events.

This approach enables:

- Loose coupling
- High cohesion
- Horizontal scalability
- Extension interoperability
- Background processing
- Reactive workflows
- Independent deployment
- Progressive capability growth

Unlike traditional request-driven architectures, the Mosaic Runtime treats events as the primary mechanism through which the platform coordinates work.

Relationship to MEG

MEG-001

↓

Engineering Principles

↓

MEG-002

↓

Runtime Behaviour

↓

Extensions

↓

Platform Features

MEG-001 defines **how software is engineered**.

MEG-002 defines **how software behaves once it is running**.

Together they establish the engineering foundation of the Mosaic platform.

Scope

This specification defines:

- Event philosophy
- Runtime lifecycle
- Event contracts
- Event naming
- Event versioning
- Publishers
- Subscribers
- Event bus
- Worker lifecycle
- Scheduling
- Retry strategy
- Idempotency
- Event ordering
- Correlation IDs
- Backpressure
- Observability
- Runtime resilience

This specification intentionally does **not** define:

- Business domains
- HTTP APIs
- Storage architecture
- Extension SDK
- Deployment
- Infrastructure

These concerns are defined by later MEG specifications.

Core Question

MEG-002 exists to answer one question.

How should independently developed capabilities coordinate work within the Mosaic Runtime?

Runtime Statement

Within Mosaic:

Capabilities publish facts. Other capabilities decide whether they care.

A capability should never need to know:

- who consumes an event
- how many consumers exist
- what work those consumers perform
- whether any consumer exists at all

This separation allows the platform to evolve without creating unnecessary coupling between capabilities.

Runtime Hierarchy

The Mosaic Runtime intentionally separates event processing into conceptual layers.

Capability

↓

Domain Event

↓

Event Bus

↓

Subscribers

↓

Background Work

↓

State Change

↓

New Events

Every layer owns exactly one responsibility.

Future chapters define each layer in detail.

Expected Outcome

After reading MEG-002 contributors should understand:

- why Mosaic is event-driven
- when events should be published
- how events are named

- how subscribers behave
- how retries work
- how failures are handled
- how background workers integrate with the runtime
- how independent extensions cooperate without direct dependencies

without discussing any individual business domain.

Repository Structure

engineering/

■■■ meg/

■■■ MEG-002 Event-Driven Runtime/

README.md

00-document-control.md

01-runtime-philosophy.md

02-why-events.md

03-event-model.md

04-event-naming.md

05-event-schema.md

06-event-versioning.md

07-event-bus.md

08-publishers.md

09-subscribers.md

10-worker-lifecycle.md

11-scheduling.md

12-idempotency.md

13-retry-strategy.md

14-event-ordering.md

15-backpressure.md

16-correlation-and-observability.md

17-runtime-shutdown.md

18-adrs.md

19-contributor-guidance.md

glossary.md

references.md

Dependencies

Required reading:

- MEG-001 Go Engineering Standards
- MDL-002 Principles
- MDL-003 Mental Model

Future companion specifications:

- MEG-003 Domain Driven Design
- MEG-004 Hexagonal Architecture
- MEG-005 Extension Platform
- MEG-006 Runtime Architecture

Design Goals

The Event-Driven Runtime is intended to produce a platform that is:

- Reactive
- Decoupled
- Observable
- Resilient
- Extensible
- Scalable
- Testable
- Deterministic
- Fault tolerant

The runtime should encourage independent capability evolution while preserving architectural consistency across the entire Mosaic ecosystem.

Review Status

Status

Draft

Owner

Lead Software Architect

Next File

00-document-control.md

Document Control

Document Information

Field	Value
Document ID	MEG-002
Title	Event-Driven Runtime
File	00-document-control.md
Status	Draft
Version	0.1
Owner	Lead Software Architect
Classification	Internal Architecture Specification

Purpose

This document establishes the governance, authority and lifecycle of the Mosaic Event-Driven Runtime specification.

MEG-002 defines the architectural standards governing how independently developed capabilities communicate and coordinate work within the Mosaic Runtime.

Unlike implementation documentation, this specification defines **runtime behaviour**, not implementation detail.

Authority

MEG-002 is the authoritative specification governing event-driven behaviour throughout the Mosaic ecosystem.

This specification applies to:

- Mosaic Core
- First-party Extensions
- Third-party Extensions
- Background Workers
- Schedulers
- Event Publishers
- Event Subscribers
- Runtime Infrastructure

Every component participating in the Mosaic Runtime **MUST** comply with the standards defined within this specification.

Relationship to Other Specifications

MEG specifications intentionally build upon one another.

MDL

↓

MDS

↓

MEG-001

↓

MEG-002

Specifically:

- **MDL** defines product philosophy.
- **MDS** defines presentation.
- **MEG-001** defines engineering practices.
- **MEG-002** defines runtime behaviour.

Later MEG specifications build upon the runtime model introduced here.

Normative Language

Unless explicitly stated otherwise, the following keywords are interpreted according to RFC 2119.

Keyword	Meaning
MUST	Mandatory requirement.
MUST NOT	Prohibited behaviour.
SHOULD	Strong recommendation. Deviation requires architectural justification.
SHOULD NOT	Discouraged except where clearly justified.
MAY	Optional behaviour based upon engineering judgement.

Examples and diagrams are informative unless explicitly identified as normative.

Runtime Principles

The Mosaic Runtime is built upon several foundational principles.

- Capabilities own behaviour.
- Events communicate facts.
- Dependencies remain explicit.
- Failures are expected.

- Components remain autonomous.
- Work is asynchronous by default.
- Runtime behaviour must be observable.
- Shutdown must be deterministic.

Every subsequent chapter expands one or more of these principles.

Document Lifecycle

MEG specifications evolve alongside the platform.

Each document progresses through the following lifecycle.

Draft

↓

Review

↓

Accepted

↓

Implemented

↓

Maintained

↓

Superseded (optional)

Accepted specifications become part of the canonical Mosaic architecture.

Historical versions SHOULD remain available for future reference.

Runtime Evolution

The runtime is expected to evolve.

However, architectural evolution should occur deliberately.

Changes affecting:

- event contracts
- event delivery guarantees
- runtime semantics
- lifecycle behaviour
- extension interaction

SHOULD be accompanied by an Architectural Decision Record (ADR).

This ensures architectural intent remains discoverable long after implementation changes.

Compliance

All runtime components SHOULD comply with MEG-002.

Where deviation becomes necessary, the repository SHOULD document:

- the reason
- expected impact
- migration strategy
- compatibility implications

Temporary deviations should eventually be removed.

Permanent deviations should generally result in updates to the specification.

Design Philosophy

MEG-002 intentionally favours:

- deterministic behaviour
- loose coupling
- eventual consistency
- explicit ownership
- bounded complexity
- observable execution

The runtime should encourage independent capability evolution without sacrificing operational clarity.

This reflects established practice in mature event-driven systems, where idempotency, observability, ordering guarantees and bounded failure handling are treated as first-class architectural concerns rather than implementation details. [oai_citation:0‡Encore Framework](https://encore.dev/articles/event-driven-architecture?utm_source=chatgpt.com)

Scope of Authority

MEG-002 governs runtime behaviour.

It does **not** define:

- business domains
- storage technologies
- transport protocols
- extension packaging
- deployment architecture

Those concerns belong to other MEG specifications.

This separation keeps runtime concerns independent from implementation concerns.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

README.md

Next File

01-runtime-philosophy.md

Runtime Philosophy

The runtime exists to orchestrate capabilities, not control them.

Purpose

The Mosaic Runtime is fundamentally different from a traditional monolithic application.

Rather than a collection of tightly coupled services calling one another directly, the runtime consists of autonomous capabilities that collaborate through events.

Each capability owns its own behaviour.

The runtime owns coordination.

Understanding this distinction is essential to understanding every architectural decision made throughout the Mosaic platform.

Philosophy

Within Mosaic:

The runtime coordinates work. Capabilities own behaviour.

The runtime should never become a place where business logic accumulates.

Instead, it provides the infrastructure that allows independently developed capabilities to cooperate safely and predictably.

Runtime Responsibilities

The runtime owns platform-wide concerns.

These include:

- Event delivery
- Capability discovery
- Lifecycle management
- Scheduling
- Background execution
- Health monitoring
- Observability
- Graceful shutdown
- Resource ownership

The runtime intentionally does **not** own:

- Media management

- Metadata
- Playback
- Authentication
- Search
- Library management

These responsibilities belong to capabilities.

Capabilities

A capability represents a self-contained piece of business functionality.

Examples include:

Metadata

Playback

Library

Search

Notifications

Recommendations

A capability should:

- own its own behaviour
- own its own state
- publish events
- subscribe to relevant events

A capability should **never** become responsible for orchestrating the wider platform.

Runtime Model

The runtime intentionally separates coordination from execution.

Capability

↓

Publishes Event

↓

Runtime

↓

Routes Event

↓

Interested Capabilities

↓

State Changes

↓

New Events

Notice that capabilities never communicate directly.

The runtime becomes the coordination layer.

The Runtime Is Not The Business

One of the easiest mistakes to make when building event-driven systems is gradually moving business logic into the event bus.

Within Mosaic this is prohibited.

The runtime should answer questions such as:

- Where should this event go?
- Who is subscribed?
- Is the runtime healthy?
- Can this work be retried?

The runtime should never answer questions such as:

- Is this movie watched?
- Should metadata be refreshed?
- Does this user have permission?

Those belong to capabilities.

Publish Facts

Capabilities publish facts.

Not commands.

Good:

MediaImported

PlaybackStarted

MetadataUpdated

Poor:

RefreshMetadata

UpdateArtwork

Facts describe what has happened.

Commands instruct another capability what to do.

Facts encourage loose coupling.

Commands create dependency.

Autonomous Capabilities

Capabilities should remain autonomous.

A capability should never need to know:

- who consumes its events
- how many consumers exist
- whether consumers exist
- what consumers do

Publishing an event should feel like writing to a public noticeboard.

Other capabilities decide whether they care.

The Runtime Owns Time

Business capabilities should not concern themselves with:

- scheduling
- retry timing
- delayed execution
- backoff
- worker allocation

Instead:

Capability

↓

Publish Event

↓

Runtime

↓

Schedule Work

↓

Deliver Event

Time is a runtime concern.

Business behaviour is not.

The Runtime Owns Reliability

Failures are inevitable.

The runtime is responsible for ensuring they do not compromise the platform.

This includes:

- retries
- dead-letter handling
- backpressure
- cancellation
- graceful shutdown

Business capabilities should focus solely on business behaviour.

Reliability is a platform responsibility.

Eventual Consistency

The Mosaic Runtime embraces eventual consistency.

Immediately after an event is published:

Capability A

↓

Event Published

↓

Capability B

↓

Capability C

↓

Capability D

Each capability progresses independently.

The platform eventually converges upon a consistent state.

Immediate synchronisation is intentionally avoided unless correctness requires it.

This aligns with common event-driven architecture principles, where autonomous components converge through asynchronous communication rather than synchronous orchestration.

([martinfowler.com](https://martinfowler.com/articles/201701-event-driven.html))(<https://martinfowler.com/articles/201701-event-driven.html>)

Progressive Capability

One of the defining characteristics of the runtime is progressive capability.

Installing an extension should not require modifying existing capabilities.

Instead:

Existing Runtime

↓

Install Extension

↓

Subscribe To Existing Events

↓

Platform Gains New Behaviour

No existing capability changes.

The runtime simply gains another participant.

This allows the platform to evolve organically.

Runtime Boundaries

The runtime deliberately exposes a small surface area.

Capabilities should depend upon:

- event publication
- event subscription
- scheduling
- lifecycle notifications

Nothing more.

Reducing the runtime API encourages long-term stability.

Runtime Does Not Own State

The runtime coordinates state changes.

It does not own business state.

Example:

Runtime

↓

MediaImported

↓

Metadata Capability

↓

Metadata Database

The runtime transports the event.

It does not inspect or modify metadata.

This distinction is critical.

Runtime Lifecycle

Every runtime component follows the same lifecycle.

Registered

↓

Initialised

↓

Started

↓

Running

↓

Stopping

↓

Stopped

Capabilities should respond to lifecycle transitions.

They should never attempt to control them.

The Runtime Is Replaceable

Business capabilities should depend upon runtime contracts.

Not runtime implementations.

This allows:

- testing
- simulation
- future runtime evolution

without requiring changes to business behaviour.

Capabilities should view the runtime as infrastructure.

Mosaic Principles

Within Mosaic:

- Capabilities own behaviour.
- The runtime owns coordination.

- Events communicate facts.
- Capabilities remain autonomous.
- The runtime owns time.
- The runtime owns reliability.
- Business state never belongs to the runtime.
- Extensions integrate through events rather than direct dependencies.

These principles define the architectural identity of the Mosaic platform.

Every future runtime decision should reinforce them.

Summary

The Mosaic Runtime exists to enable cooperation without coupling.

It coordinates.

It schedules.

It observes.

It delivers.

It never becomes the business itself.

When responsibilities remain clearly separated, new capabilities can be introduced without modifying existing ones.

That is the defining property of an extensible platform.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

00-document-control.md

Next File

02-why-events.md

Why Events?

Events are not an implementation detail. They are the language through which capabilities communicate.

Purpose

Every architectural decision carries trade-offs.

The Mosaic Runtime intentionally adopts an event-driven architecture because it aligns with the platform's core design goals:

- Extensibility
- Loose coupling
- Independent capabilities
- Background processing
- Progressive enhancement

This document explains **why** events are the primary communication mechanism within the Mosaic platform. It intentionally focuses on architectural reasoning rather than implementation details.

The Problem

Traditional applications often communicate through direct service calls.

Library

↓

Metadata

↓

Artwork

↓

Notifications

↓

Analytics

Each component becomes aware of every other component.

As functionality grows:

- dependencies multiply
- coupling increases
- testing becomes harder
- introducing new behaviour becomes increasingly expensive

The architecture gradually evolves into a dependency graph rather than a platform.

Direct Communication

Consider a seemingly simple operation.

Media Imported

A traditional implementation might become:

Library

↓

Metadata Service

↓

Artwork Service

↓

Search Service

↓

Recommendation Service

↓

Notification Service

↓

Analytics Service

The Library capability now knows about six unrelated systems.

Every future capability requires modifying Library.

This violates the Open/Closed Principle.

Event Communication

The same behaviour using events becomes:

Library

↓

MediaImported

↓

Runtime

↓

Metadata

Artwork

Search

Recommendations

Analytics

Notifications

Library publishes one fact.

Everything else becomes someone else's responsibility.

Library never changes as the platform grows.

Events Describe Facts

Within Mosaic:

Events describe something that **has already happened**.

Examples include:

MediaImported

PlaybackStarted

UserAuthenticated

LibraryScanned

These are historical facts.

They cannot be rejected.

They have already occurred.

Events Are Not Commands

Commands tell another component what to do.

GenerateArtwork

RefreshMetadata

IndexSearch

These tightly couple two capabilities.

One capability now assumes:

- another capability exists
- it performs a particular behaviour
- it should perform it immediately

This is not how Mosaic capabilities communicate.

Facts Enable Growth

Publishing facts means future capabilities require no modifications to existing systems.

Imagine adding:

AI Recommendations

No existing capability changes.

Instead:

MediaImported

↓

AI Recommendation Extension

The extension simply subscribes.

The runtime already understands how to deliver events.

This property allows Mosaic to grow indefinitely without creating dependency explosions.

Capabilities Should Not Know Each Other

One of the primary goals of Mosaic is capability independence.

A capability should never ask:

- Who needs this information?
- How many subscribers exist?
- What happens next?

Instead it should simply announce:

"This happened."

Everything afterwards belongs to the runtime.

Event Flow

Every event follows the same conceptual flow.

Business Behaviour

↓

State Changes

↓

Event Published

↓

Runtime

↓

Interested Subscribers

↓

Independent Behaviour

↓

Further Events

Notice that no capability explicitly invokes another capability.

The runtime coordinates everything.

Loose Coupling

Loose coupling produces numerous architectural benefits.

Capabilities can:

- evolve independently
- be developed independently
- be tested independently
- be replaced independently
- be disabled independently

This dramatically reduces long-term maintenance costs.

Loose coupling is one of the defining characteristics of event-driven architectures because publishers and subscribers evolve independently.

([martinfowler.com](https://martinfowler.com/articles/201701-event-driven.html))(<https://martinfowler.com/articles/201701-event-driven.html>))

Extension Independence

Extensions should integrate naturally.

Example.

Without events:

Extension

↓

Modify Core

↓

Add Service Calls

↓

Deploy

With events:

Install Extension

↓

Subscribe

↓

Receive Events

↓

Platform Expanded

The existing runtime remains unchanged.

This is one of the strongest arguments for an event-driven platform.

Failure Isolation

Suppose:

Artwork Generation

fails.

With direct calls:

Library

↓

Artwork

↓

Failure

↓

Import Fails

With events:

Library

↓

MediaImported

↓

Artwork

↓

Failure

↓

Retry Later

The import succeeds.

Artwork eventually catches up.

Independent failures remain isolated.

Asynchronous Behaviour

Many operations should not delay user interaction.

Examples include:

- metadata refresh
- artwork generation
- recommendation updates

- search indexing
- analytics
- cache warming

These naturally become background subscribers.

The initiating capability remains responsive.

Progressive Capability

The runtime encourages capabilities to appear gradually.

Initially:

Library

↓

Metadata

Later:

Library

↓

Metadata

↓

Recommendations

Later still:

Library

↓

Metadata

↓

Recommendations

↓

Collections

↓

Machine Learning

↓

Statistics

No earlier capability requires modification.

Growth becomes additive.

Not invasive.

Observable Behaviour

Events naturally produce observable systems.

Every important state transition becomes visible.

Example.

PlaybackStarted

↓

PlaybackProgress

↓

PlaybackPaused

↓

PlaybackResumed

↓

PlaybackCompleted

Operations become traceable.

Failures become diagnosable.

Observability emerges naturally from the architecture.

Event Chains

Complex workflows emerge from simple event chains.

MediaImported

↓

MetadataFetched

↓

ArtworkDownloaded

↓

MediaIndexed

↓

LibraryUpdated

↓

LibraryRendered

Each capability owns one transition.

No capability owns the entire workflow.

This keeps responsibilities small.

The Runtime Is A Coordinator

The runtime intentionally behaves like infrastructure.

It routes.

It schedules.

It retries.

It observes.

It never becomes another business capability.

This distinction keeps business logic separate from orchestration.

When Not To Use Events

Not every interaction should become an event.

Events SHOULD NOT replace:

- immediate validation
- synchronous queries
- request/response APIs
- simple in-process function calls

If one capability requires an immediate answer from another, a direct abstraction is usually more appropriate.

Events communicate facts.

They do not replace every form of communication.

Mosaic Guidelines

Within Mosaic:

- Events communicate facts.
- Capabilities SHOULD publish events after state changes.
- Capabilities SHOULD remain unaware of subscribers.
- Extensions SHOULD integrate through events.
- Direct capability dependencies SHOULD be minimised.
- Background work SHOULD originate from events where practical.
- The runtime SHOULD coordinate rather than orchestrate business behaviour.

Summary

Events are not simply another messaging mechanism.

Within Mosaic they are the language of the runtime.

They allow independently developed capabilities to cooperate without depending upon one another.

That single architectural decision enables:

- extension-first development
- progressive capability
- background processing
- resilience
- scalability
- long-term maintainability

Everything else defined within MEG-002 builds upon this principle.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

01-runtime-philosophy.md

Next File

03-event-model.md

Event Model

Events are immutable records of facts that have already occurred. They describe reality. They do not request it.

Purpose

A consistent event model is fundamental to an event-driven platform.

If every capability invents its own event structure, naming conventions and lifecycle semantics, the runtime quickly becomes fragmented.

This document defines the canonical event model for the Mosaic Runtime.

Every event published within the platform **MUST** conform to these principles.

Philosophy

Within Mosaic:

An event is an immutable fact describing a completed state transition.

Events are historical records.

Once published they cannot be modified.

If reality changes, a **new event** is published.

History is additive.

Never rewritten.

What Is An Event?

An event answers one question.

What happened?

Examples include:

MediaImported

PlaybackStarted

MetadataUpdated

ExtensionInstalled

Each describes something that has already occurred.

Nothing more.

Event Lifecycle

Every event progresses through the same lifecycle.

State Changes

↓

Event Created

↓

Published

↓

Delivered

↓

Processed

↓

Archived

Once published, an event becomes immutable.

Subscribers must treat every event as read-only.

Events Follow State

Events **MUST** be published **after** business state changes.

Correct:

Persist Media

↓

Commit Transaction

↓

Publish MediaImported

Incorrect:

Publish MediaImported

↓

Persist Media

If persistence fails, no event should exist describing work that never actually happened.

Events describe reality.

Not intention.

Events Are Immutable

Once an event has been published:

- payloads **MUST NOT** change
- timestamps **MUST NOT** change

- identifiers MUST NOT change
- metadata MUST NOT change

Corrections require a new event.

Example.

MetadataImported

↓

MetadataCorrected

Not:

Modify Existing Event

Immutability is one of the defining characteristics of event-driven systems because it preserves an accurate historical record and simplifies replay, auditing and debugging.

([martinfowler.com](https://martinfowler.com/eaDev/EventNarrative.html))(<https://martinfowler.com/eaDev/EventNarrative.html>)

Event Structure

Every runtime event consists of two parts.

Envelope

↓

Payload

The envelope describes runtime behaviour.

The payload describes business behaviour.

This separation keeps runtime infrastructure independent from domain semantics.

Event Envelope

Every event SHOULD contain standard runtime metadata.

Event ID

Event Name

Event Version

Occurred At

Correlation ID

Causation ID

Producer

Payload

These fields allow the runtime to:

- route events
- trace workflows

- correlate requests
- support replay
- diagnose failures

Future chapters define each field in detail.

Event Payload

The payload contains business information only.

Example:

MediaImported

↓

Media ID

Library ID

Source

Imported At

The payload SHOULD NOT contain:

- retry counts
- worker IDs
- routing information
- subscriber metadata

Those belong to the runtime.

Event Ownership

Every event has exactly one publisher.

Example.

Library

↓

MediaImported

Library owns the event because Library owns the state transition.

Other capabilities consume the event.

They do not redefine it.

Ownership should always be unambiguous.

Event Identity

Every event **MUST** have a globally unique identifier.

This identifier represents:

This specific occurrence of a fact.

Two identical payloads published at different times represent two different events.

Identity belongs to the occurrence.

Not the payload.

Event Time

Every event records when the business fact occurred.

Not:

- when processing completed
- when subscribers received it
- when retries succeeded

The timestamp represents reality.

Not delivery.

This distinction becomes important during replay and diagnostics.

Event Ordering

Events **SHOULD** be considered independent unless an explicit ordering guarantee exists.

Subscribers **MUST NOT** assume:

Event A

↓

Event B

will always be received in that order.

Ordering guarantees are discussed in a later chapter.

Subscribers should instead be resilient to delayed or repeated delivery.

Event Granularity

Events should describe meaningful business transitions.

Good:

PlaybackCompleted

Poor:

SetBooleanFlag

Events should communicate concepts recognised by the business domain.

Not implementation details.

Event Completeness

An event SHOULD contain sufficient information for subscribers to perform their work.

Subscribers SHOULD NOT immediately perform another query simply to understand the event.

Poor:

MediaImported

↓

Media ID Only

↓

Every Subscriber Queries Database

Better:

MediaImported

↓

Media ID

Library ID

Media Type

Source

Include enough context to minimise unnecessary coupling.

Avoid including entire domain models.

Event Independence

Events should remain independent.

Example.

MediaImported

should not contain:

Next Event

Subscriber List

Processing Instructions

The publisher describes the fact.

The runtime decides delivery.

Subscribers decide behaviour.

Each concern remains separate.

Event Size

Event payloads SHOULD remain compact.

Events are notifications.

Not object graphs.

Include:

- identifiers
- essential business context
- immutable business facts

Avoid:

- large binary data
- complete database records
- transient runtime state

Large artefacts should be referenced rather than embedded.

Event Evolution

Events evolve through versioning.

Existing events MUST remain valid.

Breaking changes require:

- new event versions
- compatibility strategy
- migration plan

Events should remain stable over time.

Future chapters define versioning standards.

Event Replay

The runtime SHOULD support replay where practical.

Replay enables:

- rebuilding projections
- recovering state
- diagnostics

- testing
- extension installation

Replay is only possible because events are immutable.

Subscribers should therefore remain deterministic.

Event Processing

Every subscriber should process events according to the same model.

Receive

↓

Validate

↓

Execute

↓

Publish New Events

Subscribers should avoid:

- modifying received events
- mutating runtime metadata
- depending upon subscriber order

Each subscriber behaves independently.

Event Chains

Events naturally create event chains.

Example.

MediaImported

↓

MetadataFetched

↓

ArtworkDownloaded

↓

LibraryIndexed

↓

RecommendationsUpdated

Each event represents a completed business transition.

No event predicts the future.

Mosaic Guidelines

Within Mosaic:

- Events **MUST** describe completed facts.
- Events **MUST** be immutable.
- Events **MUST** be published after state changes.
- Every event **MUST** have a unique identifier.
- Event payloads **SHOULD** contain business information only.
- Runtime metadata **MUST** remain outside business payloads.
- Subscribers **MUST** treat events as read-only.
- New events **MUST** be published for new facts.

Summary

The Mosaic Event Model deliberately separates:

- business facts
- runtime metadata
- delivery
- processing

This separation keeps events stable, replayable and understandable.

Every future runtime feature, including retries, observability, versioning and extension interoperability, depends upon a consistent event model.

The event model is therefore one of the foundational architectural contracts of the Mosaic Runtime.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

02-why-events.md

Next File

04-event-naming.md

Event Naming

An event name should describe a business fact so clearly that an engineer unfamiliar with the codebase immediately understands what has happened.

Purpose

Events form the language of the Mosaic Runtime.

Every capability communicates through events.

Poorly named events create confusion, coupling and ambiguity.

Well-named events create a ubiquitous language shared across:

- Core
- Extensions
- Workers
- Tooling
- Observability
- Documentation

This document defines the naming standards for every event published within the Mosaic ecosystem.

Philosophy

Within Mosaic:

Event names describe completed business facts.

An event should read naturally as a sentence.

For example:

Playback started.

becomes

PlaybackStarted

An engineer should never need additional documentation simply to understand what an event represents.

Event Grammar

Every event name follows the same grammatical structure.

<Noun><Past Tense Verb>

Examples:

MediaImported

PlaybackStarted

ArtworkDownloaded

ExtensionInstalled

This convention makes event streams readable.

Events Describe History

Events always describe something that has **already happened**.

Correct:

MediaDeleted

Incorrect:

DeleteMedia

Correct:

UserAuthenticated

Incorrect:

AuthenticateUser

Commands request work.

Events describe completed work.

Business Language

Event names should reflect business terminology.

Good:

PlaybackCompleted

Poor:

UpdatePlaybackFlag

Good:

LibraryScanned

Poor:

RunLibraryScanTask

Implementation details should never leak into event names.

Singular Events

Events describe a single occurrence.

Prefer:

MediaImported

Not:

`MediaImportedBatch`

If multiple entities are involved, the payload should describe them.

The event itself remains singular.

PascalCase

Event names **MUST** use PascalCase.

Examples:

`MetadataFetched`

`LibraryCreated`

`PlaybackResumed`

Avoid:

`metadata_fetched`

`metadataFetched`

`metadata-fetched`

Consistency improves discoverability throughout tooling and documentation.

No Technical Prefixes

Avoid implementation-specific prefixes.

Poor:

`KafkaMediaImported`

`RabbitPlaybackStarted`

`InternalMetadataUpdated`

The transport mechanism should never appear within the event name.

Events belong to the business.

Not the messaging infrastructure.

No Version Numbers

Event names **MUST** remain stable.

Avoid:

`MediaImportedV2`

`PlaybackStarted2`

Versioning belongs within the event envelope.

Not within the event name.

Future chapters define event versioning.

Avoid Generic Names

Poor:

Updated

Changed

Completed

These names communicate almost nothing.

Instead:

MetadataUpdated

PlaybackCompleted

LibraryRefreshed

The subject should always be explicit.

Avoid Internal Terminology

Suppose the implementation uses:

Projection

Internally.

The business may understand:

Library

The event should therefore be:

LibraryUpdated

Not:

ProjectionRebuilt

Event names should communicate domain concepts.

Not implementation details.

Avoid CRUD Thinking

Many systems naturally produce events such as:

UserCreated

UserUpdated

UserDeleted

While acceptable, they often reveal CRUD-centric modelling.

Prefer richer business events where appropriate.

Instead of:

MediaUpdated

Consider:

ArtworkChanged

MetadataCorrected

WatchProgressSynced

These communicate actual business meaning.

Capability Ownership

The capability owning a business concept owns its events.

Example.

Library

↓

MediaImported

Only Library should publish:

MediaImported

Other capabilities may react.

They should not redefine ownership.

Event Families

Related events naturally form families.

Example.

PlaybackStarted

PlaybackPaused

PlaybackResumed

PlaybackCompleted

PlaybackStopped

This creates a predictable event vocabulary.

New contributors quickly understand relationships.

Event Consistency

Choose one verb.

Use it everywhere.

Example.

If the platform uses:

Imported

Do not later introduce:

Loaded

for the same business concept.

Likewise:

Installed

should not later become:

Added

Consistency is more valuable than perfect wording.

Avoid Tense Changes

Events always use past tense.

Never mix:

PlaybackStarting

PlaybackStarted

PlaybackStart

Only one should exist.

Within Mosaic:

PlaybackStarted

Past tense reinforces that the event represents history.

Naming Checklist

Before introducing a new event ask:

- Does this describe a completed fact?
- Would a business stakeholder understand this?
- Is the name implementation agnostic?
- Does it follow existing terminology?
- Is it sufficiently specific?
- Does it belong to the owning capability?

If any answer is "no", reconsider the name.

Examples

Good

MediaImported

MetadataFetched
ArtworkDownloaded
PlaybackStarted
PlaybackCompleted
ExtensionInstalled
UserAuthenticated

Poor

DoImport
UpdateMedia
TaskFinished
HandlerExecuted
MessageReceived
KafkaImport
EventV2

These names expose implementation rather than business behaviour.

Event Streams

Well-named events naturally produce readable timelines.

MediaImported
↓
MetadataFetched
↓
ArtworkDownloaded
↓
LibraryIndexed
↓
RecommendationsUpdated

An engineer should understand the entire workflow simply by reading the event stream.

That is the objective.

Mosaic Guidelines

Within Mosaic:

- Event names MUST describe completed facts.
- Event names MUST use PascalCase.

- Event names **MUST** use past tense.
- Event names **MUST** represent business concepts.
- Event names **MUST NOT** contain implementation details.
- Event names **MUST NOT** include transport technology.
- Event names **MUST NOT** include version numbers.
- Related events **SHOULD** use consistent vocabulary.
- Every business concept **SHOULD** have one canonical event family.

Summary

Event names are more than identifiers.

They form the ubiquitous language of the Mosaic Runtime.

Good event names make:

- documentation easier
- observability clearer
- debugging simpler
- onboarding faster
- extension development more intuitive

Every event should read like a sentence describing something that has already become true.

That simple principle keeps the entire runtime understandable as it continues to grow.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

03-event-model.md

Next File

05-event-schema.md

Event Schema

Every event should look familiar before its payload is understood.

Purpose

A runtime containing hundreds of event types requires consistency.

Subscribers should not need to learn a different structure for every capability.

Instead, every event published within the Mosaic Runtime follows a common schema.

This document defines the canonical event schema used throughout the platform.

It intentionally separates runtime concerns from business concerns.

Philosophy

Within Mosaic:

Every event consists of an immutable runtime envelope surrounding an immutable business payload.

The runtime understands the envelope.

Business capabilities understand the payload.

Neither should depend upon the other.

Event Structure

Every event consists of two logical components.

Event

■■■ Envelope
■■■ Payload

The envelope describes:

- identity
- routing
- observability
- tracing
- compatibility

The payload describes:

- business facts

This separation allows the runtime to evolve independently from business domains.

Canonical Event

Conceptually, every event follows the same structure.

[text]
Event

```
■■■ Metadata
■
■   ■■■ Event ID
■   ■■■ Event Name
■   ■■■ Event Version
■   ■■■ Occurred At
■   ■■■ Correlation ID
■   ■■■ Causation ID
■   ■■■ Producer
■   ■■■ Trace Information
■
■■■ Payload
```

Capabilities should primarily concern themselves with the payload.

The runtime owns the metadata.

Envelope

The envelope exists for the runtime.

Business capabilities **SHOULD NOT** inspect runtime metadata unless explicitly required.

The envelope enables:

- routing
- retries
- replay
- tracing
- observability
- diagnostics

Every event contains the same envelope.

Event ID

Every event **MUST** contain a globally unique identifier.

Purpose:

- deduplication
- replay
- tracing
- diagnostics
- idempotency

The Event ID identifies the occurrence of a business fact.

Not the business entity itself.

Example.

Media ID

≠

Event ID

A media item may generate hundreds of events.

Each event receives its own identifier.

Event Name

The Event Name identifies the business fact.

Examples include:

MediaImported

PlaybackStarted

MetadataFetched

Naming standards are defined in the previous chapter.

The Event Name should remain stable across versions.

Event Version

The version identifies the payload contract.

It allows:

- backwards compatibility
- gradual migration
- extension compatibility

Versioning is discussed in detail in the next chapter.

The Event Name identifies **what** happened.

The Event Version identifies **how** the payload should be interpreted.

Occurred At

The timestamp records when the business event occurred.

Not:

- when delivered
- when processed

- when persisted
- when replayed

Time belongs to the business event.

Not the runtime.

Correlation ID

The Correlation ID links related events together.

Example.

HTTP Request

↓

MediaImported

↓

MetadataFetched

↓

ArtworkDownloaded

↓

LibraryUpdated

Every event shares the same Correlation ID.

This allows complete workflows to be reconstructed through logs and traces.

Correlation IDs are a common practice in distributed and event-driven systems because they enable end-to-end tracing across asynchronous workflows.

([opentelemetry.io](https://opentelemetry.io/docs/concepts/signals/traces/))(<https://opentelemetry.io/docs/concepts/signals/traces/>)

Causation ID

The Causation ID records:

Which event directly caused this event?

Example.

MediaImported

↓

MetadataFetched

↓

ArtworkDownloaded

MetadataFetched

↓

Causation ID

↓

MediaImported Event ID

Unlike Correlation ID:

The Causation ID changes every time a new event is produced.

Together they form an event graph.

Producer

Every event identifies the capability that produced it.

Examples.

Library

Metadata

Playback

The Producer communicates ownership.

It does **not** imply who should consume the event.

Trace Metadata

Where tracing exists, runtime metadata SHOULD include trace identifiers.

This allows:

- distributed tracing
- performance analysis
- debugging
- request reconstruction

Tracing metadata belongs entirely to the runtime.

Business payloads should remain unaware of tracing.

Payload

The payload contains immutable business facts.

Example.

MediaImported

↓

Media ID

Library ID

Source

Imported At

Payloads SHOULD contain:

- identifiers
- immutable business data
- sufficient subscriber context

Payloads SHOULD NOT contain:

- retry count
- subscriber state
- delivery status
- processing instructions

Business Context

Payloads should provide enough information for subscribers to make decisions.

Poor.

MediaImported

↓

Media ID

Every subscriber now performs another lookup.

Better.

MediaImported

↓

Media ID

Library ID

Media Type

Source

The event communicates useful business context without duplicating the entire domain model.

Payload Ownership

Only the publishing capability defines the payload.

Consumers MUST treat payloads as immutable.

Consumers SHOULD NOT assume additional fields will appear unexpectedly.

Future compatibility is achieved through versioning.

Not runtime mutation.

Event Size

Events SHOULD remain lightweight.

Good payloads contain:

- identifiers
- immutable facts
- small contextual values

Poor payloads contain:

- binary artwork
- video streams
- large metadata blobs
- complete object graphs

Large artefacts should be referenced.

Not embedded.

Event Serialization

The runtime SHOULD serialise events into a transport-neutral format.

Examples include:

- JSON
- Protocol Buffers
- MessagePack

Capabilities MUST remain unaware of serialization details.

Transport is infrastructure.

Events are business concepts.

Unknown Fields

Subscribers SHOULD ignore unknown fields where possible.

This enables:

- gradual evolution
- forward compatibility
- rolling deployments

Consumers should only depend upon fields they actually require.

Validation

Every event SHOULD be validated before publication.

Validation includes:

- required metadata
- payload completeness
- version correctness
- identifier presence

Invalid events should never enter the runtime.

Preventing invalid data is significantly cheaper than recovering from it later.

Example Conceptual Schema

Event

```
■■■ Event ID
■■■ Event Name
■■■ Version
■■■ Occurred At
■■■ Correlation ID
■■■ Causation ID
■■■ Producer
■
■■■ Payload

    ■■■ Business Identifier

    ■■■ Business Facts

    ■■■ Immutable Context
```

Notice the strict separation between runtime concerns and business concerns.

Mosaic Guidelines

Within Mosaic:

- Every event MUST contain a runtime envelope.
- Every event MUST contain a business payload.
- Envelope metadata MUST remain transport agnostic.
- Payloads MUST remain immutable.
- Event IDs MUST be globally unique.
- Correlation IDs SHOULD span complete workflows.
- Causation IDs SHOULD identify immediate parent events.
- Payloads SHOULD remain compact.
- Business payloads MUST NOT contain runtime state.

Summary

A consistent event schema allows:

- capabilities to remain independent
- runtime infrastructure to evolve
- observability to improve
- replay to function
- versioning to remain manageable
- extensions to interoperate safely

The event schema is therefore one of the most important contracts within the Mosaic Runtime.

Every capability speaks the same language because every event shares the same structure.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

04-event-naming.md

Next File

06-event-versioning.md

Event Versioning

Events are contracts. Contracts evolve deliberately, never accidentally.

Purpose

The Mosaic Runtime is designed to support long-lived capabilities that evolve independently.

Extensions may be developed by different teams.

Some may update immediately.

Others may remain unchanged for months.

Without a clear versioning strategy, even small payload changes can break unrelated capabilities.

This document defines how events evolve while preserving compatibility across the Mosaic ecosystem.

Philosophy

Within Mosaic:

Events are immutable. Contracts are versioned. Compatibility is intentional.

Breaking subscribers because an event changed is considered an architectural failure.

Publishers own compatibility.

Subscribers own resilience.

The runtime coordinates both.

Why Version Events?

Events represent contracts between independently developed capabilities.

Consider:

Library

↓

MediaImported

↓

Metadata

↓

Artwork

↓

Search

↓

Recommendations

A seemingly harmless payload modification could break every subscriber.

Versioning allows contracts to evolve safely.

Event Contracts

Every event defines a public contract.

The contract consists of:

- Event Name
- Event Version
- Payload Schema
- Business Meaning

Changing any of these may require versioning.

Event versioning is therefore an architectural concern.

Not an implementation detail.

Version Location

Event versions belong inside the runtime envelope.

Example.

Event

↓

Event Name

↓

Version

↓

Payload

Version numbers MUST NOT appear within:

MediaImportedV2

The name describes the business fact.

The version describes the payload contract.

These concerns remain separate.

Semantic Compatibility

Changes fall into three categories.

Compatible

↓

Conditionally Compatible

↓

Breaking

Only breaking changes require a new major event version.

Compatible Changes

The following changes are generally compatible.

- Adding optional fields.
- Expanding enum values where subscribers ignore unknown values.
- Improving documentation.
- Clarifying semantics without changing behaviour.

Existing subscribers should continue functioning without modification.

Conditionally Compatible Changes

Some changes require careful consideration.

Examples include:

- introducing optional nested structures
- changing validation rules
- tightening constraints
- deprecating fields

These changes may require staged rollout.

Architectural review is recommended.

Breaking Changes

Breaking changes require a new event version.

Examples include:

- removing fields
- renaming fields
- changing field meaning
- changing field type
- changing business semantics
- changing identifier behaviour

Subscribers should never unexpectedly observe these changes.

Event Evolution

Events evolve through new versions.

Example.

MediaImported

↓

Version 1

↓

Version 2

↓

Version 3

The Event Name remains constant.

Only the schema evolves.

This preserves business language while allowing implementation to mature.

Publisher Responsibilities

Publishers own event compatibility.

A publisher introducing a new version **MUST**:

- define the new schema
- document differences
- provide migration guidance
- consider existing subscribers

Compatibility is the publisher's responsibility.

Not the subscriber's.

Subscriber Responsibilities

Subscribers should process only versions they understand.

If an unsupported version is received they **SHOULD**:

- reject gracefully
- log appropriately
- avoid undefined behaviour

Subscribers should never guess the meaning of unknown contracts.

Forward Compatibility

Where practical, subscribers SHOULD ignore unknown optional fields.

Example.

Version 1

↓

Media ID

Later.

Version 2

↓

Media ID

Source

Version 1 subscribers continue operating.

They simply ignore the additional information.

Forward-compatible consumers significantly simplify rolling deployments.

Backward Compatibility

Publishers SHOULD preserve previous event versions during migration periods.

Example.

Publisher

↓

Version 1

Version 2

Subscribers migrate independently.

Eventually:

Version 1

↓

Deprecated

↓

Removed

Migration should be deliberate.

Never immediate.

Deprecation

Deprecated versions SHOULD remain supported for a defined period.

Every deprecation SHOULD include:

- announcement
- migration guidance
- removal timeline

Unexpected removal breaks trust in the runtime.

Schema Evolution

Schema evolution should favour:

Addition

↓

Deprecation

↓

Removal

Rather than:

Replacement

Gradual evolution reduces migration risk.

Event Meaning

Versioning should preserve business meaning.

Poor.

MediaImported

Version 2

↓

Now means "Media Indexed"

Business meaning changed.

A new event should exist instead.

Example.

MediaIndexed

Versioning evolves schemas.

It does not redefine history.

Runtime Behaviour

The runtime routes events regardless of version.

Subscribers determine compatibility.

Example.

MediaImported

↓

Version 2

↓

Runtime

↓

Subscriber

↓

Supports?

↓

Process

or

Reject

The runtime remains version agnostic.

Business compatibility belongs to capabilities.

Event Replay

Replay should preserve original versions.

Historical events MUST NOT be upgraded automatically.

Replay reproduces history.

It does not rewrite it.

This property makes replay trustworthy.

Documentation

Every event version SHOULD document:

- schema
- added fields
- removed fields
- semantic changes
- migration guidance

Historical documentation should remain available.

Future contributors should understand why versions changed.

Version Numbering

Event versions SHOULD use simple sequential integers.

Example.

Version 1

Version 2

Version 3

Semantic Versioning is unnecessary for individual event contracts.

The event version communicates schema evolution only.

Keeping version numbers simple reduces operational complexity.

Version Lifecycle

Every version follows the same lifecycle.

Draft

↓

Published

↓

Supported

↓

Deprecated

↓

Retired

Only supported versions should be produced by publishers.

Deprecated versions may still be consumed during migration.

Anti-Patterns

The following practices are prohibited.

Silent Schema Changes

Changing payload structure without increasing the version.

Reusing Version Numbers

Publishing different payloads using the same version.

Renaming Event Types

MediaImported

↓

MediaImportedV2

Version belongs in metadata.

Not names.

Redefining History

Changing what an existing event means.

History should remain immutable.

Automatic Payload Mutation

Converting historical events into newer schemas during replay.

Replay should reproduce reality.

Not reinterpret it.

Mosaic Guidelines

Within Mosaic:

- Every event **MUST** include a version.
- Event names **MUST** remain stable across versions.
- Breaking changes **MUST** increment the version.
- Compatible additions **SHOULD** preserve existing versions where possible.
- Publishers **MUST** own compatibility.
- Subscribers **MUST** process only supported versions.
- Historical events **MUST** remain immutable.
- Replay **MUST** preserve original versions.

Summary

Versioning allows the Mosaic Runtime to evolve without sacrificing stability.

Well-versioned events enable:

- independent capability evolution
- safe extension development
- rolling deployments

- historical replay
- long-term maintainability

Every event is a public contract.

Like every public contract, it deserves deliberate stewardship.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

05-event-schema.md

Next File

07-event-bus.md

Event Bus

The Event Bus transports facts. It never interprets them.

Purpose

The Event Bus is the central coordination mechanism of the Mosaic Runtime.

It is responsible for delivering events between capabilities while remaining completely unaware of business behaviour.

The Event Bus is **not** a workflow engine.

It is **not** an orchestration engine.

It is **not** a rules engine.

Its only responsibility is to reliably transport immutable business facts from publishers to interested subscribers.

Philosophy

Within Mosaic:

The Event Bus owns delivery. Capabilities own meaning.

The Event Bus should never understand:

- media
- playback
- metadata
- users
- extensions

Those are business concepts.

The Event Bus understands only:

- events
- subscribers
- delivery
- retries
- acknowledgements
- lifecycle

This separation is fundamental to maintaining loose coupling.

Responsibilities

The Event Bus owns the following responsibilities.

- Event publication
- Subscriber discovery
- Event routing
- Delivery guarantees
- Retry scheduling
- Dead-letter routing
- Delivery metrics
- Backpressure
- Runtime observability

The Event Bus intentionally does **not** own:

- business validation
- business workflows
- business state
- business decisions

Runtime Model

Every event follows the same runtime path.

Publisher

↓

Event Bus

↓

Subscriber A

Subscriber B

Subscriber C

↓

Independent Processing

Notice that publishers never communicate directly with subscribers.

Every interaction occurs through the Event Bus.

Publish

Publishing an event means:

This business fact has occurred.

The publisher does not ask:

- Who receives it?
- How many subscribers exist?
- Did anybody process it?
- What happens next?

Publishing should be fire-and-forget from the publisher's perspective.

Once accepted by the Event Bus, responsibility transfers to the runtime.

Subscription

Capabilities subscribe to events.

Not publishers.

Example.

Playback

↓

Subscribes

↓

MediaImported

The Library capability remains unaware that Playback exists.

Adding another subscriber never requires modifying the publisher.

Routing

The Event Bus routes events using:

Event Type

↓

Registered Subscribers

↓

Delivery

No routing logic should exist inside business capabilities.

Capabilities express interest.

The runtime performs routing.

Delivery Model

The Event Bus delivers events independently.

Example.

MediaImported

↓

Metadata

Artwork

Recommendations

Search

Analytics

Each subscriber receives the event independently.

Failure in one subscriber MUST NOT prevent delivery to others.

Subscriber isolation is one of the key properties of resilient event-driven systems.

Delivery Guarantees

Within Mosaic, the Event Bus provides:

At-Least-Once Delivery

Every published event is delivered one or more times.

Subscribers MUST therefore be idempotent.

Exactly-once delivery is intentionally avoided because it introduces substantial complexity and often relies on infrastructure-specific guarantees. At-least-once delivery with idempotent consumers is the more common architectural choice for distributed event-driven systems. ([microservices.io](https://microservices.io/post/microservices/patterns/2020/10/16/idempotent-consumer.html))

Future chapters define idempotency requirements.

Fan-Out

One event may have many subscribers.

PlaybackCompleted

↓

Statistics

↓

Recommendations

↓

History

↓

Achievements

↓

Analytics

The Event Bus performs the fan-out automatically.

Publishers remain unchanged regardless of subscriber count.

No Subscriber Ordering

Subscribers **MUST** be considered independent.

The runtime makes **no guarantee** that:

Metadata

↓

Artwork

will execute before:

Recommendations

If ordering is required, it should emerge naturally through additional events.

Never through subscriber registration order.

Runtime Registration

Capabilities register subscriptions during startup.

Runtime

↓

Capability

↓

Register Subscription

↓

Ready

Subscriptions should remain static throughout the application's lifetime unless explicitly designed otherwise.

Dynamic Extensions

Extensions register subscriptions exactly like Core capabilities.

Extension Loaded

↓

Register Events

↓

Begin Receiving

The Event Bus makes no distinction between:

- Core
- First-party extensions
- Third-party extensions

Every capability participates equally.

Delivery Independence

Suppose one subscriber fails.

MediaImported

↓

Metadata ✓

Artwork ✗

Recommendations ✓

Search ✓

The failure affects only Artwork.

Other subscribers continue normally.

Retry belongs to Artwork.

Not the publisher.

Event Acknowledgement

Subscribers acknowledge successful processing.

Receive

↓

Process

↓

Acknowledge

Only after acknowledgement is the event considered successfully processed for that subscriber.

Unacknowledged events become eligible for retry.

Event Filtering

Subscribers receive only events they have explicitly subscribed to.

Poor.

Receive Everything

↓

Filter Internally

Preferred.

Runtime

↓

Deliver Relevant Events

The runtime performs routing.

Subscribers process business logic.

Backpressure

The Event Bus is responsible for protecting the runtime during overload.

Possible strategies include:

- bounded queues
- worker pools
- rate limiting
- deferred retries

Unbounded queues are prohibited.

The runtime must remain stable under sustained load.

Future chapters discuss backpressure in detail.

Event Persistence

The Event Bus MAY persist events before delivery.

Persistence enables:

- replay
- diagnostics
- recovery
- auditing

Whether persistence is enabled is a runtime concern.

Business capabilities should remain unaware.

Event Replay

Replay follows the same delivery path as live events.

Historical Events

↓

Event Bus

↓

Subscribers

Subscribers should not need separate replay implementations.

Processing historical events should be indistinguishable from processing live events wherever practical.

Runtime Observability

Every significant Event Bus action SHOULD produce telemetry.

Examples include:

- published events
- delivery latency
- retry count
- subscriber failures
- queue depth
- dead-letter events

The Event Bus should be one of the most observable components within the platform.

If events cannot be observed, debugging distributed behaviour becomes significantly harder.

Event Bus Boundaries

The Event Bus should remain intentionally small.

It should expose behaviour such as:

- Publish
- Subscribe
- Register
- Shutdown

It should not expose business concepts.

The runtime API should remain stable even as capabilities evolve.

Anti-Patterns

The following practices are prohibited.

Business Logic Inside The Event Bus

If PlaybackCompleted

↓

Update Statistics

The Event Bus should never understand business events.

Subscriber Discovery By Publishers

Publisher

↓

Find Subscriber

↓

Call Subscriber

Publishers should never know subscribers exist.

Shared Subscriber State

Subscribers communicating through shared mutable memory rather than events.

Runtime Orchestration

The Event Bus deciding which business operation should execute next.

Business workflows emerge from events.

Not runtime decisions.

Synchronous Delivery Requirements

Publishers waiting for every subscriber before continuing.

The runtime should remain asynchronous wherever practical.

Mosaic Guidelines

Within Mosaic:

- The Event Bus **MUST** remain business agnostic.
- Publishers **MUST** publish facts only.
- Subscribers **MUST** register explicitly.
- Subscribers **MUST** remain independent.
- Delivery **MUST** be at-least-once.
- Subscriber failures **MUST NOT** affect other subscribers.

- Runtime routing MUST remain transparent to publishers.
- The Event Bus MUST remain observable.
- Extensions MUST integrate through the same Event Bus as Core capabilities.

Summary

The Event Bus is the nervous system of the Mosaic Runtime.

It transports information.

It coordinates delivery.

It observes execution.

It never owns business behaviour.

Maintaining this separation allows the platform to grow from a handful of capabilities to hundreds without introducing direct dependencies between them.

The Event Bus therefore remains one of the smallest yet most important components within the entire Mosaic architecture.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

06-event-versioning.md

Next File

08-publishers.md

Publishers

A publisher records that something happened. It never decides who should care.

Purpose

Publishers are responsible for introducing events into the Mosaic Runtime.

They represent the boundary between business behaviour and runtime coordination.

A publisher owns one responsibility:

Publish immutable business facts after successful state transitions.

Everything that follows belongs to the runtime.

This document defines the responsibilities, behaviour and constraints governing every event publisher within the Mosaic ecosystem.

Philosophy

Within Mosaic:

Publishers announce facts. They never orchestrate workflows.

Publishing an event should be the final step of completing business behaviour.

The publisher should never:

- invoke subscribers
- coordinate workflows
- wait for processing
- understand downstream behaviour

Those responsibilities belong to the runtime.

Publisher Responsibilities

Every publisher is responsible for:

- detecting completed business state changes
- constructing valid events
- publishing events
- preserving event integrity

A publisher is **not** responsible for:

- delivery
- retries

- scheduling
- subscriber discovery
- ordering
- observability

Those concerns belong entirely to the Event Bus.

Publishing Lifecycle

Every event follows the same publishing lifecycle.

Business Operation

↓

State Changes

↓

Transaction Commits

↓

Create Event

↓

Publish Event

↓

Return

Publishing should occur only after the business operation has completed successfully.

Publish After Success

Events MUST represent reality.

Correct.

Persist Metadata

↓

Commit

↓

MetadataImported

Incorrect.

MetadataImported

↓

Attempt Persist

↓

Failure

The runtime must never observe events describing work that never actually happened.

One Publisher

Every event **MUST** have exactly one publisher.

Example.

PlaybackStarted

↓

Playback Capability

Only Playback owns this business fact.

Other capabilities may observe it.

None should publish it.

Ownership must remain unambiguous.

Publisher Ownership

A capability owns the events describing the state it owns.

Examples.

Library

↓

MediaImported

Metadata

↓

MetadataFetched

Playback

↓

PlaybackCompleted

Ownership should never cross bounded contexts.

If multiple capabilities publish the same event, architectural ownership has become unclear.

Publishing Is Fire-And-Forget

Once an event has been accepted by the Event Bus:

The publisher's responsibility ends.

The publisher **MUST NOT**:

- wait for subscribers

- retry delivery
- inspect acknowledgements
- coordinate processing

Example.

Library

↓

Publish

↓

Continue

The runtime becomes responsible for reliable delivery.

Publish Facts

Publishers communicate:

What happened

Never:

What should happen

Good.

MediaImported

Poor.

GenerateArtwork

Subscribers decide what actions, if any, should follow.

Publisher Independence

Publishers must remain completely unaware of:

- subscriber count
- subscriber identity
- subscriber implementation
- subscriber success
- subscriber failure

A capability should function correctly even if no subscribers exist.

This is the defining property of loose coupling.

Constructing Events

Publishers construct complete events.

This includes:

- runtime metadata
- business payload
- identifiers
- timestamps

Incomplete events should never enter the runtime.

Validation belongs at publication time.

Publishing Transactions

Publishing should occur only after business state becomes durable.

Recommended sequence.

Business Logic

↓

Persist State

↓

Commit

↓

Publish Event

This avoids publishing events describing rolled-back work.

In distributed systems this concern is often addressed using the Transactional Outbox pattern, which records events atomically alongside business state before asynchronous publication.

([microservices.io](https://microservices.io/patterns/data/transactional-outbox.html))

Multiple Events

A single operation may publish multiple events.

Example.

Import Media

↓

MediaImported

↓

LibraryUpdated

Each event should represent a distinct business fact.

Do not combine unrelated concepts into one large event.

Event Timing

Publishers determine **when** a fact becomes true.

They do not determine:

- when subscribers execute
- when retries occur
- when work completes

The runtime controls execution timing.

Publishers control business truth.

Avoid Conditional Publishing

Poor.

If Search Enabled

↓

Publish Search Event

Better.

MediaImported

↓

Search Subscriber Exists?

↓

Runtime Delivers

Capabilities should publish business facts regardless of platform configuration.

Subscribers determine relevance.

Idempotent Publication

Publishers **SHOULD** avoid accidentally publishing duplicate events.

If duplicate publication occurs, subscriber idempotency should ensure correctness.

Publisher correctness remains preferable.

Duplicate events should represent exceptional rather than normal behaviour.

Publisher Failure

Publishing itself may fail.

Examples include:

- runtime unavailable

- event validation failure
- persistence failure

Publishers should treat publication failure as a business failure unless the runtime explicitly guarantees deferred publication.

Silent event loss is prohibited.

Event Ordering

Publishers **SHOULD** publish events in the order business facts occur.

Example.

Correct.

PlaybackStarted

↓

PlaybackCompleted

Incorrect.

PlaybackCompleted

↓

PlaybackStarted

The runtime does not infer chronology.

Publishers establish it.

Domain Boundaries

Publishers must never publish events belonging to another capability.

Poor.

Metadata

↓

PlaybackCompleted

Only Playback owns playback.

Capabilities should never invent facts outside their own domain.

Publisher Simplicity

Publishing code should remain simple.

Example.

[text]
Update State

↓

Create Event

↓

Publish

Complex business logic should already have completed.

If publishing requires extensive decision making, that logic probably belongs elsewhere.

Anti-Patterns

The following practices are prohibited.

Waiting For Subscribers

Publish

↓

Wait

↓

Continue

Publishing should not become orchestration.

Publishing Before Commit

Publishing facts before durable state exists.

Publishing Commands

RefreshMetadata

Publish facts instead.

Subscriber Discovery

Publishers attempting to locate subscribers.

Business Logic During Publication

Publication should not become another application layer.

Business decisions belong before publication.

Publishing Another Capability's Events

Every capability owns its own business facts.

Ownership should never overlap.

Mosaic Guidelines

Within Mosaic:

- Publishers **MUST** publish facts only.
- Publishers **MUST** publish after successful state transitions.
- Every event **MUST** have one canonical publisher.
- Publishers **MUST** remain unaware of subscribers.
- Publishers **MUST NOT** coordinate workflows.
- Events **MUST** be validated before publication.
- Business ownership **MUST** determine publisher ownership.
- Publication **SHOULD** remain deterministic and simple.

Summary

Publishers are intentionally uncomplicated.

They perform one task exceptionally well:

Convert completed business state into immutable events.

By remaining unaware of downstream behaviour, publishers allow the Mosaic Runtime to grow indefinitely without increasing coupling between capabilities.

This simplicity is one of the key architectural properties that enables Mosaic's extension-first platform.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

07-event-bus.md

Next File

09-subscribers.md

Subscribers

Subscribers react to facts. They never influence whether those facts occurred.

Purpose

Subscribers are responsible for reacting to events published within the Mosaic Runtime.

They represent autonomous capabilities that observe business facts and perform additional work where appropriate.

Unlike publishers, subscribers have no ownership over the event itself.

Their responsibility begins only after the event has been published.

This document defines the behaviour, responsibilities and architectural constraints governing event subscribers.

Philosophy

Within Mosaic:

Subscribers react independently to immutable business facts.

A subscriber should answer one question.

"Given this fact, is there work my capability should perform?"

Nothing more.

Subscribers should never become orchestrators.

They should simply react.

Subscriber Responsibilities

Subscribers are responsible for:

- receiving events
- validating event compatibility
- performing business behaviour
- publishing new facts
- acknowledging successful processing

Subscribers are **not** responsible for:

- event routing
- retry scheduling
- subscriber discovery

- event persistence
- delivery guarantees

Those responsibilities belong to the runtime.

Subscription Model

Subscribers explicitly declare interest.

Playback

↓

Subscribes To

↓

MediaImported

The runtime records this relationship.

Publishers remain unaware.

Processing Lifecycle

Every subscriber follows the same lifecycle.

Receive Event

↓

Validate

↓

Execute Business Logic

↓

Persist State

↓

Publish New Events

↓

Acknowledge

This lifecycle remains consistent regardless of capability.

Validate First

Before processing an event, every subscriber SHOULD validate:

- supported version
- required payload fields
- mandatory identifiers

- business preconditions

Invalid events SHOULD fail immediately.

Subscribers should never execute partial business logic against invalid contracts.

Process Independently

Subscribers should process events independently.

Example.

MediaImported

↓

Metadata

↓

Artwork

↓

Search

↓

Recommendations

Each subscriber owns its own behaviour.

No subscriber should depend upon another subscriber completing first.

Publish New Facts

Subscribers frequently become publishers.

Example.

MediaImported

↓

Metadata Subscriber

↓

MetadataFetched

The subscriber reacts.

Performs work.

Publishes a new business fact.

The chain continues.

This creates naturally evolving workflows without central orchestration.

One Responsibility

Each subscriber SHOULD own one business concern.

Good.

Artwork Subscriber

↓

Downloads Artwork

Poor.

Artwork Subscriber

↓

Downloads Artwork

Indexes Search

Calculates Recommendations

Updates Analytics

Subscribers should remain cohesive.

Idempotency

Subscribers MUST be idempotent.

Receiving the same event multiple times must produce the same final state.

Example.

MediaImported

↓

Receive Twice

↓

Metadata Exists

↓

No Duplicate State

At-least-once delivery makes idempotent subscribers a fundamental architectural requirement rather than an optimisation. The Idempotent Consumer pattern is widely recommended for this reason. ([microservices.io](https://microservices.io/post/microservices/patterns/2020/10/16/idempotent-consumer.html))

Future chapters define idempotency in detail.

Failure Isolation

Subscriber failure must remain isolated.

Example.

MediaImported

↓

Metadata ✓

↓

Artwork ✗

↓

Recommendations ✓

Artwork failure must never prevent Recommendations from processing.

The runtime manages retries independently.

Timeouts

Subscribers SHOULD honour context cancellation.

Long-running work must terminate when:

- cancellation requested
- runtime shutting down
- timeout exceeded

Subscribers should never ignore runtime lifecycle.

Retry Behaviour

Subscribers SHOULD assume events may be retried.

Business behaviour must therefore tolerate:

- duplicate delivery
- delayed delivery
- replay

Subscribers should never assume:

Receive Once

They should assume:

Receive One Or More Times

Side Effects

Subscribers frequently perform side effects.

Examples include:

- downloading artwork

- writing databases
- updating caches
- sending notifications

Side effects SHOULD occur only after successful validation.

Subscribers should avoid partially completed work wherever practical.

Ordering Assumptions

Subscribers MUST NOT assume:

PlaybackStarted

↓

PlaybackCompleted

always arrives in chronological order.

Network delays.

Retries.

Replay.

Independent processing.

All may affect delivery order.

Subscribers should validate current business state rather than relying solely on event order.

Business Ownership

Subscribers may observe many domains.

They should only modify their own.

Example.

PlaybackCompleted

↓

Recommendations

Recommendations may update recommendation state.

It should **not** modify playback history.

Every capability owns its own data.

Stateless Processing

Subscribers SHOULD remain stateless wherever practical.

State belongs in:

- databases
- caches
- repositories

Not subscriber instances.

Stateless subscribers are:

- easier to test
- easier to restart
- easier to scale

Slow Subscribers

Slow subscribers should not delay the runtime.

Possible approaches include:

- worker pools
- bounded queues
- background execution

The runtime should remain responsive even when individual capabilities perform expensive work.

Event Chaining

Subscribers naturally create event chains.

MediaImported

↓

MetadataFetched

↓

ArtworkDownloaded

↓

LibraryIndexed

Each subscriber owns one transition.

No subscriber owns the complete workflow.

Logging

Subscribers SHOULD log:

- processing failures
- unexpected states

- validation failures

Subscribers SHOULD NOT log:

- successful processing of every routine event

Routine success belongs in metrics.

Failures belong in logs.

Metrics

Every subscriber SHOULD expose:

- processing count
- failure count
- retry count
- processing latency

Subscriber metrics provide operational visibility into runtime behaviour.

Replay

Subscribers SHOULD process replayed events identically to live events.

Replay should require:

No Code Changes

Business behaviour should remain deterministic.

Historical replay should reproduce historical outcomes.

Dead-Letter Events

Subscribers SHOULD eventually give up.

Permanent failures should result in:

Retry Exhausted

↓

Dead Letter

↓

Operator Investigation

Subscribers should not retry indefinitely.

Future chapters define retry policies.

Anti-Patterns

The following practices are prohibited.

Calling Other Subscribers

Metadata

↓

Call Artwork

Publish an event instead.

Shared Subscriber State

Subscribers coordinating through mutable shared memory.

Ignoring Duplicates

Assuming every event is delivered exactly once.

Long Blocking Operations

Subscribers preventing the runtime from progressing.

Business Logic Inside Retry

Retry logic belongs to the runtime.

Business logic belongs to subscribers.

Assuming Subscriber Order

Subscribers should never rely on registration order.

Mosaic Guidelines

Within Mosaic:

- Subscribers **MUST** remain autonomous.
- Subscribers **MUST** validate events before processing.
- Subscribers **MUST** be idempotent.
- Subscribers **MUST** honour cancellation.
- Subscribers **MUST** publish new facts where appropriate.
- Subscribers **MUST** own only their own business state.
- Subscribers **MUST** tolerate duplicate and delayed delivery.

- Subscribers SHOULD remain stateless.
- Subscribers SHOULD expose operational metrics.

Summary

Subscribers transform facts into further business behaviour.

They remain independent.

They remain deterministic.

They remain unaware of one another.

This allows the Mosaic Runtime to evolve organically.

New capabilities simply subscribe to existing facts.

Existing capabilities remain unchanged.

That property is one of the defining characteristics of a truly extensible platform.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

08-publishers.md

Next File

10-worker-lifecycle.md

Worker Lifecycle

Workers execute work. The runtime owns their lifecycle.

Purpose

Workers perform the asynchronous processing that powers the Mosaic Runtime.

Every background task, scheduled operation and event subscriber ultimately executes within one or more workers.

Workers are intentionally simple.

They do not own business behaviour.

They provide an execution environment in which business behaviour can safely run.

This document defines how workers are created, managed and terminated throughout the Mosaic Runtime.

Philosophy

Within Mosaic:

Workers execute work. They never own it.

Business capabilities decide **what** should happen.

The runtime decides **where**, **when** and **how** that work executes.

Separating execution from behaviour allows the runtime to evolve independently of business capabilities.

Worker Responsibilities

Workers are responsible for:

- executing tasks
- respecting cancellation
- acknowledging completion
- reporting failures
- exposing health
- shutting down gracefully

Workers are **not** responsible for:

- scheduling work
- discovering work
- retry strategy
- orchestration

- business workflows

Those responsibilities belong elsewhere within the runtime.

Worker Lifecycle

Every worker follows the same lifecycle.

Registered

↓

Initialised

↓

Started

↓

Waiting

↓

Processing

↓

Idle

↓

Stopping

↓

Stopped

The runtime manages every lifecycle transition.

Workers should never manage themselves.

Startup

During runtime initialisation:

Runtime Starts

↓

Workers Created

↓

Dependencies Injected

↓

Subscriptions Registered

↓

Workers Started

↓

Ready

Workers should be fully initialised before accepting work.

A worker should never begin processing while partially configured.

Idle State

Workers spend most of their lifetime waiting.

Waiting

↓

Task Arrives

↓

Process

↓

Waiting

Idle workers should consume minimal resources.

Busy waiting is prohibited.

Workers should block efficiently until work becomes available.

Task Acquisition

Workers receive work from the runtime.

Runtime

↓

Queue

↓

Worker

Workers should never poll business capabilities directly.

The runtime owns work distribution.

This keeps execution infrastructure independent from business behaviour.

One Task At A Time

Unless explicitly designed otherwise, a worker processes one task at a time.

Receive

↓

Process

↓

Complete

↓

Next Task

Parallelism is achieved by increasing worker count.

Not by increasing complexity inside individual workers.

Cancellation

Every worker **MUST** honour context cancellation.

Task Running

↓

Shutdown Requested

↓

Context Cancelled

↓

Cleanup

↓

Exit

Workers should terminate promptly.

Long-running operations should periodically check:

```
[go]  
ctx.Done()
```

Ignoring cancellation is prohibited.

Graceful Shutdown

Shutdown should always be graceful.

Stop Accepting Work

↓

Finish Current Task

↓

Acknowledge

↓

Cleanup

↓

Exit

Workers should never abandon partially completed business operations unless immediate termination is unavoidable.

Graceful shutdown improves consistency and reduces unnecessary retries.

Worker Ownership

Every worker has exactly one owner.

Typically:

Runtime

↓

Worker Pool

↓

Worker

Ownership answers:

- Who started this worker?
- Who stops it?
- Who monitors it?
- Who replaces it?

Workers should never exist without a clearly defined owner.

Worker Identity

Every worker SHOULD have a unique runtime identifier.

Example.

Metadata Worker #3

Worker identity improves:

- observability
- diagnostics
- tracing
- metrics

Business capabilities should remain unaware of worker identity.

Worker identity belongs to runtime infrastructure.

Failure Handling

Worker failure should affect only the current task.

Worker

↓

Task Failure

↓

Report

↓

Retry Scheduled

↓

Continue Processing

The runtime should recover automatically wherever possible.

A single task failure should not terminate the worker.

Panic Recovery

Workers SHOULD recover from unexpected panics.

Recovery should:

- log diagnostic information
- mark the task as failed
- clean up resources
- continue serving future work

The runtime should remain resilient to isolated programming errors.

Panic recovery belongs at execution boundaries rather than within business logic.

Long Running Tasks

Some tasks naturally require extended execution.

Examples include:

- library scanning
- metadata synchronisation
- artwork generation
- cache rebuilding

These tasks should:

- expose progress
- honour cancellation
- checkpoint where practical

Workers should remain observable throughout execution.

Concurrency

Worker concurrency should be controlled by the runtime.

Example.

Queue

↓

Worker Pool

↓

Workers

↓

Results

Capabilities should never create arbitrary worker pools independently.

Runtime-managed execution keeps concurrency predictable.

Resource Ownership

Workers may temporarily own resources such as:

- database transactions
- file handles
- network connections

Workers **MUST** release every acquired resource before completing.

Resource ownership ends with task completion.

Worker Health

Every worker **SHOULD** expose runtime health.

Examples include:

- current state
- active task
- processing duration
- restart count
- failure count

Health information enables operators to diagnose runtime behaviour quickly.

Worker Pools

Workers **SHOULD** execute within managed pools.

Benefits include:

- bounded concurrency
- predictable resource usage
- simplified scheduling
- easier monitoring

Worker pools are discussed further in future runtime specifications.

Scaling

Scaling should occur by increasing workers.

Not by making workers more complex.

Preferred.

1 Task

↓

Many Simple Workers

Avoid.

1 Worker

↓

Complex Internal Scheduling

Simple workers are easier to understand and easier to replace.

Task Completion

Every completed task ends in one of four states.

Completed

Failed

Cancelled

Dead Letter

No task should disappear silently.

Every outcome should be observable.

Restart Behaviour

Workers SHOULD be restartable without affecting business correctness.

Because:

- business state belongs elsewhere

- events are immutable
- subscribers are idempotent

Restarting a worker should simply continue processing available work.

This property significantly improves operational resilience.

Worker Metrics

Every worker SHOULD expose:

- tasks processed
- task duration
- queue wait time
- failures
- retries
- cancellations
- utilisation

Worker metrics provide insight into runtime capacity and bottlenecks.

Anti-Patterns

The following practices are prohibited.

Self-Managing Workers

Workers creating or destroying other workers.

Hidden Goroutines

Workers spawning unmanaged background goroutines.

Ignoring Cancellation

Workers continuing indefinitely after shutdown begins.

Infinite Loops Without Blocking

Workers consuming CPU while waiting for work.

Business Scheduling

Workers deciding when future work should execute.

Scheduling belongs to the runtime.

Permanent Mutable State

Workers retaining business state between tasks.

Business state belongs to capabilities.

Mosaic Guidelines

Within Mosaic:

- Workers **MUST** remain execution infrastructure.
- Workers **MUST** honour cancellation.
- Workers **MUST** release acquired resources.
- Workers **SHOULD** process one task at a time.
- Worker pools **SHOULD** be runtime managed.
- Workers **MUST** expose health and metrics.
- Workers **MUST** recover gracefully from unexpected failures.
- Workers **MUST** remain stateless wherever practical.
- Every worker **MUST** have a clearly defined owner.

Summary

Workers are the execution engine of the Mosaic Runtime.

They provide a predictable, observable and resilient environment in which business behaviour executes.

By separating execution from business logic, the runtime gains the freedom to:

- scale independently
- recover automatically
- evolve scheduling strategies
- improve observability
- optimise resource usage

Business capabilities remain focused on business behaviour.

The runtime remains focused on execution.

That separation is one of the key architectural principles underpinning the Mosaic platform.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

09-subscribers.md

Next File

11-scheduling.md

Scheduling

Time is a runtime concern. Business capabilities should describe what must happen, never when it should happen.

Purpose

Not all work should execute immediately.

Some work should occur:

- after a delay
- at a specific time
- periodically
- after a timeout
- once another condition has been satisfied

Within Mosaic, these concerns belong to the runtime scheduler.

Business capabilities should never implement their own scheduling logic.

Instead, they describe work.

The runtime determines when that work should execute.

Philosophy

Within Mosaic:

Capabilities own intent. The runtime owns time.

Time is infrastructure.

Business logic should remain completely unaware of:

- timers
- cron expressions
- polling loops
- scheduling algorithms
- worker allocation

This separation allows scheduling behaviour to evolve independently of business behaviour.

Why Scheduling Exists

Many runtime operations are naturally delayed.

Examples include:

- Metadata refresh
- Cache invalidation
- Periodic library scanning
- Extension health checks
- Retry backoff
- Cleanup tasks
- Token expiration
- Scheduled notifications

These operations share one common characteristic.

They depend upon **time**, not user interaction.

Runtime Model

Every scheduled task follows the same lifecycle.

Capability



Request Schedule



Runtime Scheduler



Execution Time



Worker



Business Behaviour



Events

Notice that the capability never manages the timer itself.

Scheduling Responsibilities

The runtime scheduler owns:

- delayed execution
- recurring execution
- retry scheduling
- timeout scheduling

- worker allocation
- schedule persistence
- cancellation
- observability

It intentionally does **not** own:

- business decisions
- task implementation
- workflow orchestration

Business Responsibilities

Capabilities may request scheduling.

They should never implement scheduling.

Example.

Good.

Metadata Capability

↓

Request Refresh In 24 Hours

Poor.

Metadata Capability

↓

Sleep 24 Hours

↓

Refresh

Business logic should remain independent of time.

One-Time Scheduling

One-time tasks execute once.

Examples include:

Refresh Metadata

↓

Tomorrow

Expire Invitation

↓

24 Hours

Retry Download

↓

30 Seconds

Once complete, the schedule is removed.

Recurring Scheduling

Recurring tasks execute repeatedly.

Examples include:

Library Scan

↓

Every 6 Hours

Health Check

↓

Every Minute

Metrics Snapshot

↓

Every 30 Seconds

Recurring schedules remain active until explicitly cancelled.

Delayed Execution

Some work should intentionally occur later.

Example.

PlaybackStopped

↓

Schedule History Sync

↓

30 Seconds Later

Delaying work can:

- reduce unnecessary processing
- batch operations
- improve responsiveness

The runtime owns these decisions.

Retry Scheduling

Retries are simply scheduled work.

Failure

↓

Retry Requested

↓

Scheduler

↓

Worker

↓

Retry

Subscribers should never implement retry loops.

Retries belong to the runtime.

Future chapters define retry policies.

Cron Jobs

Traditional cron jobs are discouraged.

Poor.

Cron

↓

Poll Database

↓

Look For Work

Preferred.

Event

↓

Schedule

↓

Execute

The runtime should react to business events rather than continually polling for changes.

Periodic schedules remain appropriate for genuine maintenance work.

Schedule Ownership

Every scheduled task has exactly one owner.

Example.

Metadata Capability

↓

Metadata Refresh Schedule

Only the owning capability should create or cancel its schedules.

The runtime executes them.

It does not define them.

Schedule Identity

Every scheduled task **SHOULD** have a unique identifier.

Identity enables:

- cancellation
- diagnostics
- metrics
- replay
- observability

The scheduler should treat schedules as first-class runtime objects.

Persistence

Long-running schedules **SHOULD** survive runtime restarts.

Examples include:

- recurring scans
- subscription renewals
- maintenance tasks

Ephemeral schedules may remain in memory.

Persistent schedules should be restored during startup.

The runtime owns persistence.

Capabilities remain unaware.

Cancellation

Schedules **SHOULD** be cancellable.

Typical lifecycle.

Schedule

↓

Waiting

↓

Cancelled

Cancellation should:

- release resources
- prevent future execution
- remain observable

Cancellation should never silently disappear.

Scheduling Events

The scheduler **SHOULD** publish runtime events.

Examples include:

TaskScheduled

TaskExecuted

TaskCancelled

TaskExpired

These are **runtime events** rather than business events.

They improve observability without coupling business capabilities to scheduler internals.

Scheduling Precision

Not every scheduled task requires millisecond precision.

Examples.

High precision:

- Playback synchronisation
- Session expiration

Low precision:

- Metadata refresh
- Library scan
- Cleanup

The scheduler should optimise for correctness rather than unnecessary precision.

Resource Management

Scheduling should remain bounded.

The runtime should avoid:

- unlimited pending schedules
- duplicate recurring schedules
- abandoned timers

Every scheduled task consumes runtime resources.

Ownership must remain explicit.

Restart Behaviour

Following a restart:

Runtime

↓

Restore Persistent Schedules

↓

Resume Execution

Business capabilities should not need to recreate long-lived schedules manually.

The runtime restores them.

Observability

The scheduler SHOULD expose:

- active schedules
- completed schedules
- cancelled schedules
- execution latency
- queue depth
- missed executions

Scheduling should remain one of the most observable runtime components.

Scaling

Scheduling decisions should remain centralised.

Execution should remain distributed.

Scheduler

↓

Workers

↓

Capabilities

The scheduler decides **when**.

Workers decide **where**.

Capabilities decide **what**.

This separation keeps responsibilities clear.

Anti-Patterns

The following practices are prohibited.

Sleeping Inside Business Logic

```
[go]  
time.Sleep(...)
```

Business capabilities should never delay themselves.

Infinite Polling

```
for {  
    check()  
    sleep()  
}
```

Polling should be replaced with events wherever practical.

Self-Scheduling Capabilities

Capabilities creating their own timer infrastructure.

Scheduling belongs to the runtime.

Hidden Timers

Background timers started automatically during object construction.

All scheduling should remain explicit.

Duplicate Schedules

Multiple recurring schedules performing identical work.

The runtime should detect and prevent unnecessary duplication.

Scheduling Business Decisions

The scheduler should never determine:

Should Metadata Refresh?

It simply executes requested work.

Business decisions belong to capabilities.

Mosaic Guidelines

Within Mosaic:

- The runtime **MUST** own scheduling.
- Business capabilities **MUST** remain time agnostic.
- Retries **MUST** be scheduled by the runtime.
- Persistent schedules **SHOULD** survive restarts.
- Schedules **MUST** be observable.
- Schedules **MUST** be cancellable.
- Polling **SHOULD** be replaced with events wherever practical.
- Workers **MUST** execute scheduled work.
- The scheduler **MUST** remain business agnostic.

Summary

Scheduling is infrastructure.

It allows capabilities to express **intent** without becoming responsible for **time**.

By separating scheduling from business behaviour, the Mosaic Runtime gains:

- deterministic execution
- simplified capabilities
- improved observability
- graceful recovery
- scalable worker allocation

Time becomes another service provided by the platform.

Capabilities remain focused entirely on business behaviour.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

[10-worker-lifecycle.md](#)

Next File

[12-idempotency.md](#)

Idempotency

Every event should be safe to process more than once. Correctness should never depend upon perfect delivery.

Purpose

The Mosaic Runtime guarantees **at-least-once delivery**.

This means every subscriber must assume an event may be delivered:

- once
- multiple times
- after a delay
- after a retry
- during replay

Without idempotency, duplicate delivery produces duplicate side effects.

This document defines how capabilities ensure repeated event processing always converges on the same final state.

Philosophy

Within Mosaic:

Processing an event twice should produce the same final business state as processing it once.

Idempotency is not an optimisation.

It is a fundamental property of every event subscriber.

Correctness must never depend upon receiving an event exactly once.

Why Idempotency Exists

Consider:

MediaImported

↓

Metadata Subscriber

↓

Metadata Database

Suppose the subscriber:

- downloads metadata
- stores metadata

- crashes before acknowledging the event

The runtime retries.

Without idempotency:

Download

↓

Store

↓

Download

↓

Store

Duplicate work.

Potentially duplicate state.

Idempotency prevents this.

At-Least-Once Delivery

The Mosaic Runtime deliberately provides:

Publish

↓

One Or More Deliveries

↓

Acknowledged

Not:

Exactly Once

Exactly-once delivery generally requires distributed coordination between the event bus, storage engine and subscriber, introducing significant complexity. Mosaic instead adopts the more common architectural approach of at-least-once delivery with idempotent consumers. ([microservices.io](https://microservices.io/post/microservices/patterns/2020/10/16/idempotent-consumer.html))

Idempotent Behaviour

An operation is idempotent when:

Execute

↓

State A

and

Execute

↓

Execute Again

↓

State A

The final state remains identical.

Repeated execution produces no additional business effect.

Business State

Idempotency concerns business state.

Not implementation.

Poor.

Handler executed twice

Good.

Library contains one media item.

The implementation may execute multiple times.

The business result should remain correct.

Event Identity

Every event contains a unique Event ID.

Subscribers SHOULD use this identifier when determining whether work has already been completed.

Example.

Receive Event

↓

Event ID Seen?

↓

Yes

↓

Ignore

OR

No

↓

Process

↓

Record Event ID

The Event ID represents one occurrence of a business fact.

Natural Idempotency

Some operations are naturally idempotent.

Example.

Set Status = Completed

Running repeatedly produces:

Completed

No additional work occurs.

Prefer naturally idempotent business operations wherever practical.

Artificial Idempotency

Other operations require explicit tracking.

Example.

Send Email

Running twice sends two emails.

Instead.

Receive Event

↓

Already Sent?

↓

No

↓

Send

↓

Record Completion

Artificial idempotency introduces explicit duplicate detection.

Business Keys

Sometimes Event IDs are insufficient.

Example.

Library

↓

Media ID

If only one metadata record should ever exist per media item:

Media ID

↓

Metadata Exists?

↓

Skip

Business identifiers frequently provide stronger guarantees than event identifiers alone.

Database Constraints

Whenever possible, correctness should be enforced by persistence.

Examples include:

- unique constraints
- primary keys
- upserts

Example.

INSERT

↓

Conflict

↓

Update

Database constraints provide an additional layer of protection against duplicate processing.

Business correctness should not rely solely upon application code.

Upserts

Prefer:

Insert Or Update

Rather than:

Insert

↓

Duplicate

↓

Failure

Upserts naturally support idempotent behaviour.

They simplify subscriber implementation.

Event Recording

Subscribers MAY maintain a processed-event store.

Example.

Event ID

↓

Processed?

↓

Execute

↓

Record

This approach is particularly useful for side-effect-heavy operations.

The runtime does not require a specific implementation.

Only the resulting behaviour.

Side Effects

Special care is required for external side effects.

Examples include:

- email
- notifications
- webhooks
- external APIs

Repeated execution may produce:

- duplicate emails
- duplicate notifications
- duplicate API calls

Subscribers should therefore verify whether the side effect has already occurred before repeating it.

Event Replay

Replay intentionally delivers historical events again.

Replay should therefore produce:

Historical Event

↓

Subscriber

↓

Current State Remains Correct

Replay should never corrupt business state.

This property depends entirely upon idempotent subscribers.

Retries

Retries become trivial when subscribers are idempotent.

Failure

↓

Retry

↓

Failure

↓

Retry

↓

Success

Subscribers do not need to distinguish between:

- original delivery
- retry
- replay

They simply process events safely.

Event Ordering

Idempotency should not rely upon ordering.

Suppose:

PlaybackCompleted

arrives before:

PlaybackStarted

Subscribers should validate current business state rather than assuming chronological delivery.

Ordering guarantees belong elsewhere.

Idempotency remains independent.

Compensating Events

Business state should never be "rolled back" by replay.

Instead:

MetadataImported

↓

MetadataCorrected

New facts supersede previous facts.

History remains intact.

Subscribers simply converge upon current truth.

Stateless Subscribers

Stateless subscribers naturally encourage idempotency.

State should remain in:

- repositories
- databases
- projections

Not subscriber instances.

Restarting a subscriber should not affect correctness.

Observability

Duplicate processing SHOULD remain observable.

Useful metrics include:

- duplicate events ignored
- replayed events processed
- idempotency failures
- constraint violations

Operational visibility helps identify architectural problems before they affect users.

Anti-Patterns

The following practices are prohibited.

Assuming Exactly Once

Receive

↓

Execute

↓

Never Again

Subscribers must always assume duplicate delivery.

Side Effects Before Validation

Send Email

↓

Check Duplicate

Validation should always precede external effects.

Mutable Event History

Changing historical events to avoid duplicate processing.

History remains immutable.

Subscribers adapt.

Runtime-Owned Idempotency

The runtime should not decide business correctness.

Subscribers own idempotent behaviour.

Ignoring Duplicate Delivery

Assuming retries will never occur.

Duplicate delivery is expected.

Not exceptional.

Mosaic Guidelines

Within Mosaic:

- Every subscriber **MUST** be idempotent.
- Duplicate event delivery **MUST** produce the same business state.
- Business correctness **MUST NOT** depend upon exactly-once delivery.
- Event IDs **SHOULD** support duplicate detection.

- Database constraints SHOULD reinforce idempotency.
- External side effects MUST be protected against duplication.
- Replay MUST remain safe.
- Retries MUST assume duplicate execution.

Relationship to the Runtime

Idempotency is one of the architectural properties that allows the Mosaic Runtime to remain simple.

Because subscribers are idempotent:

- retries become inexpensive
- worker crashes become recoverable
- replay becomes possible
- rolling deployments become safer
- runtime coordination becomes dramatically simpler

Rather than building an increasingly complicated runtime attempting to prevent duplicates, Mosaic accepts duplicate delivery and requires subscribers to handle it correctly.

This significantly improves resilience while reducing runtime complexity.

Summary

Idempotency is not about preventing duplicate execution.

It is about ensuring duplicate execution produces correct behaviour.

Within the Mosaic Runtime, correctness is achieved through:

- immutable events
- idempotent subscribers
- durable business state
- explicit ownership

When these principles are followed, retries, replay and recovery become natural properties of the platform rather than exceptional situations requiring special handling.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

[11-scheduling.md](#)

Next File

[13-retry-strategy.md](#)

Retry Strategy

Retries exist to recover from transient failures. They should never become an infinite substitute for fixing permanent ones.

Purpose

Failure is an expected characteristic of distributed systems.

Networks fail.

Services become temporarily unavailable.

Databases restart.

Extensions crash.

The purpose of the Mosaic Runtime is not to eliminate failure.

It is to recover from transient failure safely while exposing permanent failure clearly.

This document defines how retries are performed throughout the Mosaic Runtime.

Philosophy

Within Mosaic:

Retry infrastructure belongs to the runtime. Recovery belongs to the capability.

Business capabilities should never implement retry loops.

Instead they should:

- return failures
- remain idempotent
- allow the runtime to decide when, whether and how work should be retried.

Why Retries Exist

Consider:

MetadataDownloaded

↓

Store Metadata

↓

Database Connection Lost

The business operation has not failed permanently.

The database was temporarily unavailable.

Retrying later is entirely reasonable.

Now consider:

MediaImported

↓

Invalid Media Identifier

Retrying will never succeed.

The failure is permanent.

Understanding this distinction is fundamental.

Failure Categories

Failures fall into two categories.

Transient

↓

Retry

Permanent

↓

Fail

The runtime should never blindly retry every failure.

Transient Failures

Transient failures are expected to recover naturally.

Examples include:

- temporary network failure
- service unavailable
- database restart
- timeout
- rate limiting
- temporary file lock

Retries are appropriate.

Permanent Failures

Permanent failures cannot succeed through repetition.

Examples include:

- invalid payload

- unsupported event version
- missing required fields
- business validation failure
- corrupted data

Retries should not occur.

These failures should be surfaced immediately.

Retry Lifecycle

Every retry follows the same lifecycle.

Failure

↓

Retry Eligible?

↓

Yes

↓

Schedule Retry

↓

Worker Executes

↓

Success

or

Retry Again

or

Dead Letter

Retry is therefore another form of scheduled work.

Not a special execution path.

Runtime Ownership

The runtime owns:

- retry scheduling
- retry timing
- retry counting
- retry cancellation
- retry observability

Capabilities own:

- determining whether the operation succeeded
- returning meaningful failures
- remaining idempotent

Responsibilities remain clearly separated.

Exponential Backoff

Retries SHOULD use exponential backoff.

Example.

1 second

↓

2 seconds

↓

4 seconds

↓

8 seconds

↓

16 seconds

Exponential backoff reduces:

- cascading failures
- unnecessary load
- resource contention

Immediate retries often amplify outages.

Exponential backoff with optional jitter is widely recommended to avoid synchronized retry storms.
([aws.amazon.com])(<https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/>)

Jitter

Retry timing SHOULD include random jitter.

Without jitter:

10,000 retries

↓

Exactly 30 seconds later

↓

10,000 more requests

With jitter:

10,000 retries

↓

Distributed across time

This significantly reduces retry storms.

Maximum Retries

Retries MUST be bounded.

Example.

Retry 1

Retry 2

Retry 3

Retry 4

↓

Dead Letter

Infinite retries are prohibited.

Every retry strategy must eventually terminate.

Retry Budget

The runtime SHOULD maintain retry budgets.

Budgets prevent one failing capability from consuming disproportionate runtime resources.

Examples include:

- maximum retries
- maximum retry duration
- maximum concurrent retries

Resource usage should remain predictable.

Retry Delay Ownership

Capabilities should never decide retry timing.

Poor.

```
[go]  
time.Sleep(30 * time.Second)
```

Better.

```
Return Error
```

↓

Runtime Schedules Retry

Time remains a runtime concern.

Business logic remains pure.

Idempotency Requirement

Retries assume idempotent subscribers.

Without idempotency:

Retry

↓

Duplicate Business State

With idempotency:

Retry

↓

Correct Final State

Retry safety depends entirely upon the previous chapter.

Retry Classification

Errors SHOULD communicate retry intent.

Conceptually.

Retryable

or

Permanent

The runtime should not inspect error messages.

Capabilities should classify failures explicitly.

Future runtime APIs should expose this distinction clearly.

Retry Observability

Every retry SHOULD produce telemetry.

Examples include:

- retry count
- retry latency
- retry success
- retry exhaustion

- retry cancellation

Operators should understand:

- why retries occurred
- how frequently
- whether they succeeded

Retries should never become invisible.

Retry Cancellation

Retries should respect runtime shutdown.

Retry Scheduled

↓

Shutdown Requested

↓

Retry Cancelled

The runtime should never execute retries after shutdown begins unless explicitly configured to resume them after restart.

Retry Persistence

Pending retries SHOULD survive runtime restarts.

Example.

Retry Scheduled

↓

Runtime Stops

↓

Runtime Starts

↓

Retry Resumed

Retry scheduling should be durable wherever business correctness depends upon eventual completion.

Dead Letter Transition

Retries eventually terminate.

Retry Exhausted

↓

Dead Letter

↓

Operator Investigation

Retries are intended to recover temporary failures.

They are not intended to conceal permanent ones.

Immediate Retry

Immediate retry SHOULD be rare.

Appropriate examples include:

- optimistic locking conflicts
- short-lived resource contention

Even then:

Retry count should remain bounded.

Retry Transparency

Business capabilities should not know:

- current retry number
- retry delay
- retry scheduling algorithm

They simply process events.

The runtime manages delivery behaviour.

Retry Independence

Each subscriber owns its own retry lifecycle.

Example.

MediaImported

↓

Metadata Retry

↓

Artwork Success

↓

Search Success

Subscriber retries should never block unrelated capabilities.

Failure isolation remains intact.

Retry Metrics

The runtime SHOULD expose:

- retries scheduled
- retries completed
- retries cancelled
- retries exhausted
- average retry latency
- retry queue depth

Retry metrics provide early warning of degrading platform health.

Anti-Patterns

The following practices are prohibited.

Infinite Retry Loops

Retry Forever

Sleeping Inside Subscribers

```
[go]  
time.Sleep(...)
```

Retrying Validation Failures

Invalid business input should fail immediately.

Subscriber-Owned Retry Logic

Retry belongs to runtime infrastructure.

Retrying Without Idempotency

Duplicate execution must always be safe.

Retry Storms

Large numbers of simultaneous retries without:

- backoff
- jitter
- budgets

Mosaic Guidelines

Within Mosaic:

- Retries **MUST** remain runtime responsibilities.
- Retries **MUST** use exponential backoff.
- Retries **SHOULD** include jitter.
- Retries **MUST** remain bounded.
- Subscribers **MUST** remain idempotent.
- Retry scheduling **SHOULD** survive restart where required.
- Retry behaviour **MUST** remain observable.
- Permanent failures **MUST NOT** be retried.
- Retry exhaustion **MUST** transition to dead-letter handling.

Relationship to the Runtime

Retry is simply another form of scheduling.

The runtime already owns:

- time
- workers
- execution

Retry naturally becomes another scheduled execution.

This keeps business capabilities free from infrastructure concerns while allowing the runtime to evolve retry strategies without modifying business code.

Summary

Retries should recover temporary failure.

They should never disguise permanent failure.

Within Mosaic, retries are:

- runtime managed
- bounded
- observable
- idempotent
- deterministic

When combined with idempotent subscribers and immutable events, retries become a routine operational mechanism rather than a source of architectural complexity.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

12-idempotency.md

Next File

14-event-ordering.md

Event Ordering

Correct systems do not depend upon perfect ordering. They depend upon correct business state.

Purpose

Distributed systems naturally introduce uncertainty.

Events may be:

- delayed
- duplicated
- replayed
- processed concurrently
- delivered out of chronological order

The Mosaic Runtime intentionally avoids guaranteeing global event ordering because doing so significantly reduces scalability and increases coupling.

Instead, capabilities are engineered to converge upon correct business state regardless of delivery order wherever practical.

This document defines the ordering guarantees provided by the Mosaic Runtime and the responsibilities of capabilities consuming events.

Philosophy

Within Mosaic:

Ordering is a business concern, not a transport guarantee.

The runtime transports events.

Business capabilities determine whether order is significant.

When ordering is required, it should be modelled explicitly rather than assumed implicitly.

Why Ordering Is Difficult

Consider two events.

PlaybackStarted

↓

PlaybackCompleted

Business chronology is obvious.

Runtime chronology is not.

A temporary network delay may produce:

PlaybackCompleted

↓

PlaybackStarted

Both events are valid.

Only their arrival differs.

Subscribers must therefore distinguish between:

- event occurrence
- event delivery

These are different concepts.

Occurrence vs Delivery

Every event has two timelines.

Business Timeline

↓

Occurred

Runtime Timeline

↓

Delivered

The runtime preserves occurrence metadata.

It does not guarantee delivery order.

Subscribers should therefore reason about business time rather than arrival time.

Runtime Guarantees

The Mosaic Runtime guarantees:

- every accepted event will eventually be delivered (subject to retry policy)
- every event preserves its metadata
- every event preserves its occurrence timestamp

The runtime does **not** guarantee:

- global ordering
- subscriber ordering
- cross-capability ordering
- simultaneous delivery

These guarantees intentionally remain minimal.

Ordering Domains

Ordering is meaningful only within a bounded business context.

Example.

Playback

↓

PlaybackStarted

↓

PlaybackPaused

↓

PlaybackCompleted

Ordering matters.

However:

PlaybackCompleted

↓

MetadataFetched

There is usually no meaningful ordering relationship.

Capabilities should define ordering only where the business genuinely requires it.

Global Ordering

Global ordering is prohibited.

Example.

Every Event

↓

Single Global Queue

↓

One Worker

While simple, this approach:

- limits throughput
- increases latency
- couples unrelated capabilities
- reduces resilience

The runtime intentionally avoids this architecture.

Independent work should remain independent.

Per-Entity Ordering

Where ordering is required, it SHOULD generally be scoped to a business entity.

Example.

Media

↓

MediaImported

↓

MetadataFetched

↓

ArtworkDownloaded

Ordering exists because all events concern the same media item.

This provides deterministic behaviour without requiring global coordination.

Subscriber Expectations

Subscribers MUST assume:

- duplicate delivery
- delayed delivery
- reordered delivery

Subscribers SHOULD NOT assume:

Receive

↓

Chronological

Instead they should validate current business state before performing work.

State Validation

Suppose:

PlaybackCompleted

arrives first.

The subscriber should ask:

Current Playback State?

Rather than:

Did PlaybackStarted arrive?

Business state is authoritative.

Event order is informative.

Sequence Numbers

Some event families benefit from sequence numbers.

Example.

Playback

↓

Sequence 1

↓

Sequence 2

↓

Sequence 3

Sequence numbers allow subscribers to detect:

- missing events
- duplicates
- stale events

They should be introduced only where ordering genuinely matters.

Most events do not require them.

Version vs Sequence

Do not confuse:

Version

with

Sequence

Version identifies:

Schema

Sequence identifies:

Business chronology

They solve entirely different problems.

Concurrent Events

Independent capabilities may legitimately publish events simultaneously.

Example.

MediaImported

↓

MetadataFetched

↓

ArtworkDownloaded

and

MediaImported

↓

SearchIndexed

Neither workflow depends upon the other.

The runtime should allow maximum concurrency.

Artificial ordering reduces scalability.

Ordering Through Events

When business ordering genuinely exists, model it explicitly.

Example.

MetadataFetched

↓

ArtworkRequested

↓

ArtworkDownloaded

Each event naturally establishes the next business step.

Ordering emerges through business facts rather than runtime configuration.

Replay Ordering

Replay SHOULD preserve original occurrence order where practical.

However:

Subscribers must still remain resilient to duplicate processing and partial replay.

Replay correctness should never depend solely upon transport order.

Business state remains authoritative.

Event Timestamps

Subscribers SHOULD use:

Occurred At

when chronological reasoning is required.

Subscribers SHOULD NOT use:

Received At

Delivery time reflects runtime behaviour.

Occurrence time reflects business reality.

Missing Events

Subscribers should tolerate missing events gracefully.

Example.

PlaybackCompleted

↓

PlaybackStarted Missing

Possible responses include:

- query current state
- ignore stale event
- log diagnostic information
- request reconciliation

Subscribers should never enter undefined behaviour because one expected event failed to arrive.

Stale Events

Events may become stale.

Example.

MetadataFetched

↓

Several Hours Later

↓

MetadataCorrected

Receiving the older event later should not overwrite newer business state.

Subscribers should compare:

- timestamps
- versions
- current state

before applying changes.

Causal Ordering

Where business causality matters, subscribers SHOULD use:

- Correlation ID
- Causation ID

rather than delivery order.

These identifiers describe business relationships.

Transport order does not.

Runtime Behaviour

The runtime intentionally avoids reordering events.

It delivers events as they become available.

Capabilities determine whether ordering is relevant.

This keeps the runtime:

- simple
- scalable
- transport agnostic

Business semantics remain outside runtime infrastructure.

Anti-Patterns

The following practices are prohibited.

Assuming FIFO Globally

Event A

↓

Event B

↓

Always Delivered In Order

Subscriber Registration Order

Assuming subscriber execution order has business meaning.

Delivery Time As Business Time

Using:

Received At

instead of:

Occurred At

Blocking Unrelated Work

Preventing independent capabilities from progressing while waiting for ordered processing.

Runtime-Owned Business Ordering

The runtime should never determine:

Playback

↓

Metadata

↓

Search

Business ordering belongs to capabilities.

Mosaic Guidelines

Within Mosaic:

- Global ordering **MUST NOT** be assumed.
- Subscribers **MUST** tolerate out-of-order delivery.
- Business state **MUST** remain authoritative.
- Ordering **SHOULD** be scoped to business entities where required.
- Sequence numbers **SHOULD** only be introduced when justified.
- Event timestamps **SHOULD** represent business occurrence.
- Correlation and causation **SHOULD** express business relationships.
- Runtime delivery **MUST** remain independent of business semantics.

Relationship to the Runtime

Relaxing ordering guarantees is one of the key architectural decisions enabling Mosaic's scalability.

Because the runtime does not attempt to globally sequence every event:

- worker pools remain highly parallel
- extensions remain independent
- failures remain isolated
- throughput scales naturally

- capabilities evolve independently

Where ordering matters, business capabilities model it explicitly.

Where it does not, the runtime remains free to optimise for concurrency.

Summary

Perfect ordering is an attractive illusion.

In distributed systems it often comes at the expense of scalability, resilience and simplicity.

Within Mosaic, the runtime delivers immutable business facts.

Capabilities determine how those facts should influence business state.

By treating ordering as a business concern rather than a runtime guarantee, the platform remains both highly concurrent and architecturally simple.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

13-retry-strategy.md

Next File

15-backpressure.md

Backpressure

A healthy runtime knows when to slow down. An unhealthy runtime tries to do everything at once.

Purpose

The Mosaic Runtime is designed to operate continuously under varying workloads.

During normal operation, events flow smoothly through the platform.

However, temporary spikes are inevitable.

Examples include:

- importing a large media library
- installing a new extension
- rebuilding metadata
- replaying historical events
- recovering after downtime

Without backpressure, these bursts can overwhelm:

- workers
- databases
- storage
- external APIs
- memory
- CPU

This document defines how the Mosaic Runtime controls workload to maintain predictable behaviour under load.

Philosophy

Within Mosaic:

The runtime should slow work down before it breaks.

Throughput is desirable.

Stability is mandatory.

When resources become constrained, the runtime should reduce intake rather than consume unlimited memory or spawn unlimited work.

What Is Backpressure?

Backpressure is the mechanism through which the runtime communicates:

"I cannot safely accept more work right now."

Rather than continuing to accept events indefinitely, the runtime intentionally limits throughput to protect overall system health.

Backpressure is a fundamental concept in reactive systems because it allows producers and consumers to operate at sustainable rates instead of overwhelming one another.

([reactivemanifesto.org](https://www.reactivemanifesto.org/))

Why Backpressure Matters

Consider:

Library Scan

↓

50,000 MediaImported Events

↓

Metadata

↓

Artwork

↓

Search

↓

Recommendations

If every event immediately creates work:

- memory usage grows
- queues expand indefinitely
- workers become saturated
- external providers become rate limited

Eventually:

The runtime becomes unstable.

Backpressure prevents this.

Runtime Model

The runtime continuously balances:

Incoming Work

↓

Queues

↓

Workers

↓

Completed Work

If workers cannot keep pace:

Queues grow.

Eventually the runtime begins applying backpressure.

Backpressure Principles

The runtime should always prefer:

Slow Processing

↓

Stable Runtime

Rather than:

Unlimited Throughput

↓

Resource Exhaustion

Graceful degradation is always preferable to catastrophic failure.

Bounded Queues

Every runtime queue **MUST** have a maximum capacity.

Queue

↓

Maximum Size

↓

Backpressure

Unbounded queues are prohibited.

Unlimited buffering simply converts CPU pressure into memory pressure.

Eventually the runtime still fails.

Queue Ownership

Each capability owns its own work queue.

Example.

Metadata Queue

Artwork Queue

Search Queue

Recommendation Queue

Independent queues prevent slow capabilities from blocking unrelated work.

Failure isolation remains intact.

Worker Saturation

Suppose:

Metadata

↓

All Workers Busy

New work should queue.

If the queue becomes full:

Backpressure should be applied.

Workers should never grow without limit.

Resource Protection

Backpressure protects:

- CPU
- memory
- database pools
- blob storage
- network connections
- external APIs

Protecting infrastructure is one of the runtime's primary responsibilities.

Business capabilities should remain unaware.

Producer Behaviour

Publishers should never attempt to bypass runtime backpressure.

Poor.

Queue Full

↓

Retry Immediately

↓

Queue Full

↓

Retry Again

This amplifies overload.

Instead:

Runtime

↓

Signals Backpressure

↓

Retry Scheduled

The runtime controls recovery.

Queue Growth

Queue depth should remain observable.

Typical lifecycle.

Empty

↓

Growing

↓

Healthy

↓

Near Capacity

↓

Backpressure

↓

Recovery

The runtime should begin protecting itself before queues become completely full.

Priority

Not all work is equally important.

Examples.

High Priority.

- Playback
- Authentication
- User interaction

Lower Priority.

- Metadata refresh
- Recommendation generation
- Analytics

The scheduler MAY prioritise critical work during sustained load.

Business correctness should always take precedence over convenience.

Load Shedding

Some work may be safely discarded.

Examples include:

- duplicate refresh requests
- repeated health checks
- obsolete cache refreshes

Other work must never be discarded.

Examples include:

- playback progress
- authentication events
- library imports

The runtime should understand this distinction.

Coalescing

Repeated equivalent work SHOULD be combined where practical.

Example.

Poor.

LibraryUpdated

↓

Refresh Cache

↓

LibraryUpdated

↓

Refresh Cache

↓

LibraryUpdated

↓

Refresh Cache

Better.

LibraryUpdated

↓

Queue Refresh

↓

Additional Requests

↓

Single Refresh

The runtime should avoid redundant work whenever correctness permits.

Rate Limiting

External systems frequently impose rate limits.

Examples include:

- TMDb
- AniList
- Docker APIs
- Remote storage

Backpressure should naturally integrate with rate limiting.

Rather than failing continuously, the runtime should reduce throughput to sustainable levels.

Extension Isolation

Extensions should never be capable of overwhelming the runtime.

Suppose:

Third-Party Extension

↓

Produces Millions Of Events

The runtime should:

- isolate the extension
- apply backpressure
- preserve Core functionality

Core stability always takes precedence over extension throughput.

Recovery

Backpressure should be temporary.

Load Spike

↓

Backpressure

↓

Queues Drain

↓

Normal Operation

Recovery should occur automatically.

Manual intervention should rarely be required.

Metrics

The runtime SHOULD expose:

- queue depth
- worker utilisation
- rejected work
- deferred work
- queue wait time
- average processing latency

These metrics provide early warning of runtime stress.

Operators should identify overload before users experience degraded behaviour.

Adaptive Scaling

Where supported, the runtime MAY increase worker capacity during sustained load.

However:

Scaling should remain bounded.

Unlimited worker creation simply moves the bottleneck elsewhere.

Scaling should complement backpressure.

Not replace it.

Circuit Breakers

Backpressure integrates naturally with circuit breakers.

Suppose:

TMDB Offline

Instead of:

Retry

Retry

Retry

Retry

The runtime should:

Open Circuit

↓

Reduce Requests

↓

Recover Gradually

This protects both the runtime and external dependencies.

Replay

Replay should respect normal backpressure.

Historical replay should never bypass:

- queues
- workers
- rate limits
- resource limits

Replay should appear identical to live processing from the runtime's perspective.

Anti-Patterns

The following practices are prohibited.

Unlimited Queues

Append Forever

Unlimited Workers

Queue Full

↓

Create Worker

↓

Repeat Forever

Busy Waiting

Workers repeatedly polling empty queues.

Ignoring Resource Limits

Continuing to accept work after resource exhaustion.

Extension Starvation

Allowing one extension to consume all runtime capacity.

Immediate Retry During Overload

Retries should reduce pressure.

Not increase it.

Mosaic Guidelines

Within Mosaic:

- Every queue **MUST** be bounded.
- Every worker pool **MUST** be bounded.
- The runtime **MUST** apply backpressure before resource exhaustion.
- Queue depth **MUST** remain observable.
- Extensions **MUST** be isolated from one another.
- Recovery **SHOULD** occur automatically.
- Replay **MUST** respect runtime limits.
- High-priority work **SHOULD** remain responsive during overload.
- Stability **MUST** always take precedence over throughput.

Relationship to the Runtime

Backpressure is the mechanism that keeps the Mosaic Runtime stable under unpredictable workloads.

Combined with:

- worker pools
- scheduling
- retries
- idempotency
- event isolation

it allows the platform to remain responsive even during significant operational stress.

Rather than treating overload as an exceptional condition, the runtime treats it as an expected characteristic of a long-running system.

This philosophy produces a platform that fails gracefully rather than catastrophically.

Summary

The purpose of backpressure is not to process more work.

It is to process work sustainably.

Within Mosaic, backpressure ensures:

- predictable resource usage
- resilient extensions
- protected infrastructure
- graceful degradation
- operational stability

The runtime should always prefer temporary slowdown over permanent failure.

That single principle keeps the platform healthy as it grows from a handful of capabilities to hundreds of independently developed extensions.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

14-event-ordering.md

Next File

16-correlation-and-observability.md

Correlation and Observability

A distributed system cannot be understood by reading code alone. It must explain itself while it is running.

Purpose

As the Mosaic Runtime grows, individual business operations will span multiple capabilities, workers and extensions.

Consider a simple user action.

Play Media

↓

Playback

↓

Progress Tracking

↓

Recommendations

↓

Analytics

↓

Notifications

By the time the operation completes, dozens of events may have been published.

Without observability, understanding what actually happened becomes extremely difficult.

This document defines how the Mosaic Runtime provides complete visibility into distributed execution.

Philosophy

Within Mosaic:

Every significant runtime action should be traceable from beginning to end.

The runtime should never become a black box.

Operators should be able to answer:

- What happened?
- Why did it happen?
- What caused it?
- What failed?
- How long did it take?
- Which capability was responsible?

without modifying production code.

Observability Pillars

The Mosaic Runtime adopts the three pillars of observability.

Logs

+

Metrics

+

Traces

Each provides different information.

Together they explain runtime behaviour.

Modern observability platforms, including OpenTelemetry, are built around these three signal types.
([opentelemetry.io](https://opentelemetry.io/docs/concepts/observability-primer/))

Correlation

Correlation links related work together.

Suppose a user imports media.

MediaImported

↓

MetadataFetched

↓

ArtworkDownloaded

↓

LibraryUpdated

↓

RecommendationsGenerated

Although many capabilities participate, they all belong to one business operation.

Correlation allows the runtime to reconstruct that workflow.

Correlation ID

Every workflow begins with a Correlation ID.

Example.

HTTP Request

↓

Correlation ID Created

↓

Every Event

↓

Same Correlation ID

The Correlation ID remains constant throughout the lifetime of the workflow.

It identifies:

One business operation.

Not one event.

Causation ID

Correlation explains:

What belongs together?

Causation explains:

What directly caused this event?

Example.

MediaImported

↓

MetadataFetched

↓

ArtworkDownloaded

ArtworkDownloaded

↓

Causation ID

↓

MetadataFetched

Each event identifies its immediate parent.

Together:

Correlation creates workflows.

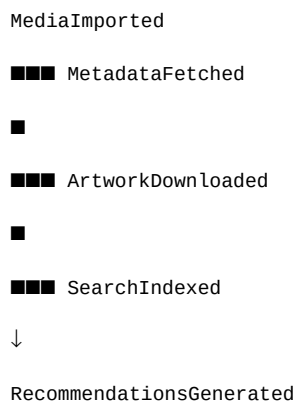
Causation creates dependency graphs.

Event Graph

Combining Correlation and Causation naturally produces an event graph.

Import Library

↓



This graph allows operators to understand exactly how work propagated through the runtime.

Distributed Tracing

Every event **SHOULD** participate in distributed tracing.

Tracing answers questions such as:

- Which capability was slow?
- Which worker executed the task?
- Which dependency failed?
- Where was latency introduced?

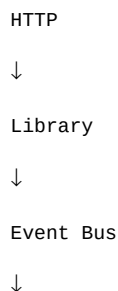
Tracing should span:

- HTTP
- Workers
- Event Bus
- Extensions
- External APIs

OpenTelemetry provides a vendor-neutral standard for propagating tracing context across distributed systems. ([opentelemetry.io](https://opentelemetry.io/docs/concepts/signals/traces/))

Trace Context

The runtime **SHOULD** automatically propagate trace context.



Metadata

↓

Artwork

↓

Search

Business capabilities should not manually manage trace propagation.

The runtime owns tracing infrastructure.

Structured Logging

Every runtime component SHOULD emit structured logs.

Example.

[text]

Timestamp

Level

Correlation ID

Causation ID

Capability

Worker

Event

Message

Logs should be machine-readable.

Not free-form text.

Structured logging enables reliable filtering, aggregation and analysis.

Logging Philosophy

Logs should explain:

- unexpected behaviour
- failures
- lifecycle transitions
- operational decisions

Logs should **not** describe routine successful execution.

Poor.

Processed Event

10,000 times.

Good.

Retry exhausted.

Moving event to dead letter queue.

Routine behaviour belongs in metrics.

Unexpected behaviour belongs in logs.

Metrics

Metrics describe system health.

Every capability SHOULD expose:

- events processed
- processing latency
- failures
- retries
- queue depth
- throughput

Metrics answer:

How healthy is the platform?

They do not explain individual requests.

Traces

Traces explain one workflow.

Metrics explain all workflows.

Logs explain exceptional workflows.

Each serves a different purpose.

Metrics

↓

Platform Health

Traces

↓

Single Workflow

Logs

↓

Unexpected Behaviour

Together they form complete observability.

Runtime Events

The runtime itself should publish operational events.

Examples include:

`WorkerStarted`

`WorkerStopped`

`RetryScheduled`

`RetryExhausted`

`BackpressureApplied`

`ExtensionLoaded`

These are runtime events.

Not business events.

They describe platform behaviour.

Capability Visibility

Every capability SHOULD expose:

- health
- version
- subscriptions
- queue depth
- worker utilisation
- processing latency

Capabilities should become observable without requiring custom instrumentation.

Health

Health should answer:

Can this capability currently perform work?

Examples.

Healthy.

`Ready`

Degraded.

`External API unavailable`

Unhealthy.

`Database disconnected`

Health should represent operational readiness.

Not merely process existence.

Correlation Across Extensions

Third-party extensions participate exactly like Core.

Example.

MediaImported

↓

Core

↓

Third-Party Extension

↓

Analytics Extension

Every event shares the same Correlation ID.

The runtime should make extension boundaries invisible during tracing.

Failure Investigation

Suppose artwork generation fails.

Operators should be able to answer:

- Which media?
- Which worker?
- Which extension?
- Which retry?
- Which API?
- Which event?
- Which workflow?

without enabling additional debugging.

Observability should already contain the answer.

Event Timeline

A single Correlation ID should naturally produce a timeline.

MediaImported

↓

MetadataFetched

↓

ArtworkRequested

↓

ArtworkDownloaded

↓

LibraryUpdated

Reading the timeline should explain the entire workflow.

No additional context should be required.

Runtime Instrumentation

Instrumentation belongs to the runtime.

Capabilities should not manually:

- propagate trace IDs
- generate correlation IDs
- publish metrics
- manage log formats

Business logic should remain unaware of operational infrastructure wherever practical.

Privacy

Observability **MUST** respect privacy.

Logs and traces **SHOULD NOT** contain:

- passwords
- access tokens
- API keys
- encryption secrets
- personally identifiable information unless explicitly required

Correlation should identify workflows.

Not expose sensitive data.

Anti-Patterns

The following practices are prohibited.

Plain Text Logs

Something happened...

Without structured metadata.

Missing Correlation IDs

Events that cannot be connected to their originating workflow.

Manual Trace Propagation

Business capabilities managing tracing state directly.

Logging Successful Every Operation

Generating excessive log volume for normal behaviour.

Metrics Without Context

Counters that cannot be attributed to capabilities or event types.

Runtime Blindness

Components executing work without exposing:

- health
- metrics
- traces

Mosaic Guidelines

Within Mosaic:

- Every workflow **MUST** have a Correlation ID.
- Every event **SHOULD** include a Causation ID.
- Structured logging **MUST** be used throughout the runtime.
- Metrics **SHOULD** describe platform health.
- Traces **SHOULD** describe workflow execution.
- Runtime instrumentation **SHOULD** be automatic.
- Extensions **MUST** participate in runtime observability.
- Sensitive information **MUST NOT** appear in logs or traces.
- Every significant runtime action **SHOULD** be observable.

Relationship to the Runtime

Observability is not an operational feature added after development.

It is part of the runtime architecture.

By making:

- events
- workers
- scheduling
- retries
- queues
- extensions

fully observable, the Mosaic Runtime becomes significantly easier to:

- operate
- debug
- optimise
- extend

Observability therefore enables confidence as much as it enables diagnostics.

Summary

An event-driven platform cannot rely upon stack traces and debugger sessions alone.

Its behaviour emerges from many independent capabilities working together.

The purpose of observability is therefore simple:

Make the invisible visible.

Every event.

Every worker.

Every retry.

Every extension.

Every workflow.

The runtime should always be able to explain itself.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

15-backpressure.md

Next File

17-runtime-shutdown.md

Runtime Shutdown

Stopping a distributed system is as important as starting one. Shutdown is not the absence of work. It is the controlled completion of work.

Purpose

Every long-running system must eventually stop.

Shutdown may occur because of:

- deployment
- upgrades
- maintenance
- scaling
- configuration changes
- operating system signals
- unexpected failures

Within the Mosaic Runtime, shutdown is considered a first-class architectural concern.

Stopping the platform should never:

- lose events
- corrupt state
- abandon work
- leave resources allocated

This document defines how the Mosaic Runtime performs graceful shutdown.

Philosophy

Within Mosaic:

The runtime should stop accepting new work before stopping existing work.

Graceful shutdown is the process of allowing in-flight operations to complete while preventing additional work from entering the system.

The objective is deterministic behaviour.

Not immediate termination.

Shutdown Principles

Every shutdown should satisfy the following principles.

- No new work accepted.
- Existing work allowed to finish where practical.
- Context cancellation propagated.
- Resources released.
- Runtime state remains consistent.
- Shutdown remains observable.

Every component participates.

No component should define its own shutdown semantics.

Runtime Lifecycle

The runtime follows a predictable lifecycle.

Starting

↓

Running

↓

Stopping

↓

Stopped

Only the runtime transitions between lifecycle states.

Capabilities react to transitions.

They do not control them.

Shutdown Sequence

Graceful shutdown follows the same sequence every time.

Shutdown Requested

↓

Stop Accepting New Events

↓

Cancel Scheduling

↓

Drain Queues

↓

Finish Active Work

↓

Release Resources

↓

Stop Workers

↓

Runtime Stops

The order is deliberate.

Changing it risks inconsistent behaviour.

Initiating Shutdown

Shutdown begins when the runtime receives a termination request.

Examples include:

- SIGTERM
- SIGINT
- administrator request
- orchestrator termination
- controlled restart

The runtime immediately transitions into:

Stopping

No additional work should be accepted after this point.

Stop Accepting New Work

The first responsibility of shutdown is preventing new work.

Examples include:

- HTTP listeners stop accepting requests
- event publishers reject new publications
- schedulers stop creating future work
- extension loading stops

Existing work continues.

New work does not begin.

Scheduler Behaviour

During shutdown the scheduler SHOULD:

Stop Scheduling

↓

Cancel Future Timers

↓

Persist Outstanding Schedules

Recurring schedules should not continue creating work once shutdown begins.

Long-lived schedules may be restored after restart.

Queue Draining

Existing queues should drain naturally.

Queue

↓

Workers Continue

↓

Queue Empty

Workers should continue processing available tasks until:

- queues empty
- cancellation deadline reached

Queue draining reduces unnecessary retries after restart.

Worker Shutdown

Workers receive cancellation through context.

Runtime

↓

Cancel Context

↓

Worker Cleanup

↓

Worker Exit

Workers should:

- finish current work where practical
- acknowledge completed work
- release resources
- terminate promptly

Workers should never ignore cancellation indefinitely.

Active Tasks

Long-running tasks should periodically check:

```
[go]  
ctx.Done()
```

If cancellation occurs they should:

- clean up
- return a meaningful status
- avoid partial state where practical

Business correctness should always take precedence over speed of shutdown.

Event Delivery

Events already accepted by the Event Bus should continue through the delivery pipeline where practical.

The runtime SHOULD avoid abandoning accepted events.

If completion is impossible:

- work should remain durable
- processing should resume after restart

Shutdown should never silently lose accepted work.

Retry Queue

Pending retries SHOULD be persisted.

Example.

Retry Scheduled

↓

Shutdown

↓

Persist

↓

Restart

↓

Retry Continues

Retries should survive controlled restarts.

Business correctness depends upon eventual execution.

Extension Shutdown

Extensions participate in runtime shutdown exactly like Core capabilities.

Lifecycle.

Runtime



Shutdown Notification



Extension Cleanup



Extension Stops

Extensions should never require special shutdown handling.

The runtime should treat all capabilities equally.

Resource Cleanup

Every runtime component is responsible for releasing owned resources.

Examples include:

- database connections
- file handles
- network sockets
- timers
- worker pools
- subscriptions

Ownership determines cleanup responsibility.

Resources should never be released by unrelated components.

Timeouts

Graceful shutdown should remain bounded.

Example.

Shutdown



30 Second Deadline



Graceful Completion

↓

Forced Termination

The runtime should not wait indefinitely.

Eventually the platform must terminate.

The timeout should be configurable.

Forced Shutdown

If graceful shutdown exceeds its deadline:

Timeout

↓

Force Termination

Forced shutdown is a last resort.

It may:

- interrupt work
- require retries
- delay eventual consistency

The runtime should make every reasonable attempt to avoid this outcome.

Observability

Shutdown should be fully observable.

Useful events include:

RuntimeStopping

QueueDrained

WorkerStopped

ExtensionStopped

RuntimeStopped

Operators should understand:

- how long shutdown required
- whether work completed
- whether work was abandoned
- whether retries were persisted

Shutdown should never appear mysterious.

Health During Shutdown

During graceful shutdown:

Health should report:

Not Ready

before:

Stopped

This allows:

- load balancers
- orchestrators
- service discovery

to stop routing new work before termination.

The runtime remains alive.

It simply stops accepting additional work.

Startup Symmetry

Startup and shutdown should mirror one another.

Startup

↓

Initialise

↓

Register

↓

Start

↓

Ready

Shutdown

↓

Stop Accepting

↓

Drain

↓

Cleanup

↓

Stopped

Symmetrical lifecycle management simplifies reasoning about the runtime.

Crash Recovery

Unexpected crashes differ from graceful shutdown.

After restart:

Recover Durable State

↓

Restore Schedules

↓

Resume Queues

↓

Restart Workers

↓

Continue Processing

Business correctness should depend upon durability.

Not graceful shutdown always succeeding.

Testing Shutdown

Graceful shutdown SHOULD be tested.

Examples include:

- worker cancellation
- queue draining
- retry persistence
- extension cleanup
- scheduler shutdown

Shutdown is part of normal runtime behaviour.

It deserves the same engineering attention as startup.

Anti-Patterns

The following practices are prohibited.

Immediate Process Exit

Shutdown

↓

`os.Exit()`

without cleanup.

Ignoring Cancellation

Workers continuing indefinitely after shutdown begins.

Accepting New Work During Shutdown

Shutdown

↓

Continue Accepting Events

Resource Leaks

Failing to close:

- database pools
- timers
- subscriptions
- workers

Silent Shutdown

Stopping without exposing runtime events or metrics.

Capability-Owned Shutdown

Business capabilities deciding when the runtime should terminate.

Lifecycle belongs to the runtime.

Mosaic Guidelines

Within Mosaic:

- Shutdown **MUST** be graceful.
- New work **MUST** stop before existing work.
- Workers **MUST** honour cancellation.
- Queues **SHOULD** drain where practical.

- Pending retries SHOULD survive restart.
- Resource ownership MUST determine cleanup.
- Shutdown SHOULD remain observable.
- Graceful shutdown SHOULD remain bounded by timeout.
- Capabilities MUST react to runtime lifecycle rather than define it.

Relationship to the Runtime

Graceful shutdown completes the runtime lifecycle defined throughout MEG-002.

Combined with:

- worker ownership
- scheduling
- retries
- idempotency
- observability

shutdown becomes a predictable operational process rather than an emergency procedure.

This consistency allows the Mosaic Runtime to:

- deploy safely
- scale safely
- recover safely
- evolve safely

without compromising business correctness.

Summary

Stopping a platform should be as carefully engineered as starting it.

Within Mosaic, graceful shutdown ensures:

- accepted work completes
- future work pauses
- resources are released
- state remains consistent
- recovery remains possible

A runtime that cannot stop predictably cannot be considered production ready.

Graceful shutdown is therefore not merely an operational concern.

It is a core architectural property of the Mosaic Runtime.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

16-correlation-and-observability.md

Next File

18-adrs.md

Architectural Decision Records (ADRs)

The runtime is one of the most stable parts of the Mosaic platform. Changes to its behaviour should be deliberate, documented and historically traceable.

Purpose

The Mosaic Runtime forms the foundation upon which every capability and extension operates.

Changes to runtime behaviour have platform-wide consequences.

Architectural Decision Records (ADRs) ensure that significant runtime decisions are:

- documented
- reviewed
- understood
- discoverable

long after the original implementation has changed.

Philosophy

Within Mosaic:

Every significant runtime behaviour should exist because of a documented architectural decision.

The runtime should never evolve accidentally.

Future contributors should understand:

- why a decision exists
- what alternatives were considered
- what trade-offs were accepted
- how the decision affects the wider platform

without reading implementation code.

Why Runtime ADRs Matter

Unlike ordinary application code, runtime behaviour affects:

- every capability
- every extension
- every worker
- every scheduler
- every deployment

Changing runtime semantics without documentation creates architectural drift.

ADRs preserve intent.

When An ADR Is Required

Runtime ADRs SHOULD be created whenever a decision changes:

- event semantics
- delivery guarantees
- runtime lifecycle
- scheduling behaviour
- retry behaviour
- ordering guarantees
- extension interaction
- worker lifecycle
- shutdown semantics
- observability contracts

If the decision changes how the runtime behaves, it probably deserves an ADR.

Examples

The following runtime decisions SHOULD each have their own ADR.

ADR-001

Reactive Runtime

ADR-002

At-Least-Once Delivery

ADR-003

Idempotent Subscribers

ADR-004

Event Envelope

ADR-005

Runtime Scheduling

ADR-006

Worker Lifecycle

ADR-007

Backpressure Strategy

ADR-008

Extension Integration Model

These decisions influence the architecture of every capability built upon the runtime.

Runtime Stability

The runtime is intentionally conservative.

Changes should be rare.

Every runtime change affects:

- backwards compatibility
- extension compatibility
- operational behaviour
- developer expectations

Architectural stability is therefore prioritised over rapid evolution.

ADR Structure

Every runtime ADR SHOULD contain:

Title

↓

Status

↓

Context

↓

Problem

↓

Options

↓

Decision

↓

Consequences

↓

Migration

↓

Related Specifications

Runtime decisions frequently require migration guidance.

This section should therefore be mandatory.

Context

The Context section should describe:

- existing runtime behaviour
- operational constraints
- architectural limitations
- motivating problems

Readers unfamiliar with the platform should understand why the decision became necessary.

Options

Every significant runtime decision SHOULD document reasonable alternatives.

Example.

Exactly Once

↓

At Least Once

↓

At Most Once

Rejected options are valuable architectural knowledge.

They explain why the runtime behaves as it does today.

Decision

The Decision section should state:

What behaviour becomes the new runtime standard?

The wording should be concise.

Implementation belongs elsewhere.

Consequences

Every runtime decision introduces trade-offs.

Example.

Choosing:

At Least Once

Produces:

Benefits.

- simpler runtime
- greater resilience
- better scalability

Costs.

- idempotent subscribers required
- duplicate delivery possible

Every ADR should acknowledge both.

Architecture is always compromise.

Migration

Runtime behaviour changes frequently require migration.

Migration guidance should explain:

- affected capabilities
- extension impact
- compatibility strategy
- rollout expectations

Migration planning should exist before implementation begins.

Runtime Compatibility

Runtime ADRs should consider compatibility across:

- Core
- First-party extensions
- Third-party extensions
- Development tooling
- Operational tooling

The runtime exists for the entire ecosystem.

Not merely Core.

Cross References

Runtime ADRs SHOULD reference:

- MEG specifications
- related ADRs
- affected capabilities

- extension specifications

Example.

ADR-007

↓

MEG-002

↓

Section 15

Backpressure

Architecture should remain navigable.

Knowledge should not become fragmented.

Superseding Runtime Decisions

The runtime will evolve.

When replacing behaviour:

ADR-005

↓

Superseded By

↓

ADR-014

History should remain intact.

Previous reasoning remains valuable even after behaviour changes.

Repository Structure

Recommended layout.

architecture/

adrs/

ADR-001-reactive-runtime.md

ADR-002-event-delivery.md

ADR-003-worker-lifecycle.md

ADR-004-runtime-scheduling.md

ADR-005-extension-runtime.md

Runtime ADRs should remain close to the architectural specifications governing them.

Review Process

Runtime ADRs SHOULD receive architectural review before acceptance.

Reviewers should evaluate:

- correctness
- scalability
- operational impact
- compatibility
- long-term maintenance
- alignment with MEG

Runtime behaviour should never change through implementation alone.

Architecture should lead implementation.

Documentation

Every accepted runtime ADR SHOULD eventually be reflected within:

- MEG specifications
- runtime documentation
- contributor guidance
- implementation comments where appropriate

The MEG defines current standards.

ADRs explain how those standards emerged.

Mosaic Guidelines

Within Mosaic:

- Significant runtime decisions SHOULD have ADRs.
- Runtime ADRs MUST explain architectural reasoning.
- Alternatives MUST be documented.
- Trade-offs MUST be acknowledged.
- Migration guidance SHOULD accompany runtime changes.
- Historical ADRs MUST remain available.
- Runtime behaviour SHOULD never change without corresponding architectural documentation.

Relationship to MEG-002

MEG-002 defines:

How the runtime behaves today.

Runtime ADRs explain:

Why it behaves that way.

Together they create a complete architectural record.

Future contributors should rarely need to rediscover runtime design decisions through experimentation.

Instead, the reasoning should already exist.

Summary

The Mosaic Runtime is the platform's execution engine.

Changes to its behaviour affect every capability.

Documenting those changes is therefore an architectural responsibility rather than a documentation exercise.

Well-maintained ADRs preserve not only implementation history, but architectural intent.

That intent is one of the most valuable assets of any long-lived software platform.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

17-runtime-shutdown.md

Next File

19-contributor-guidance.md

Contributor Guidance

The runtime belongs to the platform. Every contribution should strengthen its predictability, resilience and simplicity.

Purpose

The Mosaic Runtime is shared infrastructure.

Every capability, extension and application depends upon it behaving consistently.

Unlike ordinary business code, runtime changes affect the entire ecosystem.

This document provides practical guidance for engineers contributing to the Event-Driven Runtime.

It explains how contributors should apply the architectural principles established throughout MEG-002.

Philosophy

Within Mosaic:

Protect the runtime before extending it.

New functionality should never compromise:

- determinism
- observability
- resilience
- compatibility
- simplicity

The runtime should evolve carefully.

Capabilities should evolve rapidly.

Before Modifying The Runtime

Every contributor SHOULD understand:

- the runtime philosophy
- existing event contracts
- lifecycle semantics
- worker ownership
- retry behaviour
- scheduling model
- observability expectations

Runtime changes should never begin with implementation.

They should begin with understanding.

Before Introducing A New Event

Ask:

- Does this describe a completed business fact?
- Does an equivalent event already exist?
- Which capability owns this event?
- Does this event belong in the business domain or the runtime?

Duplicate events should be avoided.

Business language should remain consistent.

Before Creating A Runtime Event

Runtime events describe platform behaviour.

Examples include:

`WorkerStarted`

`RetryScheduled`

`BackpressureApplied`

Business events describe domain behaviour.

Examples include:

`PlaybackStarted`

`MediaImported`

Contributors should distinguish clearly between these two categories.

Before Adding A Subscriber

Ask:

- Does this capability genuinely need the event?
- Can it remain independent?
- Is the subscriber idempotent?
- Does it own the resulting business state?

Subscribers should remain autonomous.

Before Publishing An Event

Confirm:

- Business state has committed.
- Payload is complete.
- Metadata is correct.
- Event ownership is clear.
- Event naming follows MEG-002.
- The event represents a completed fact.

Events should never describe work that might happen.

Only work that has happened.

Before Introducing Scheduling

Ask:

- Is this genuinely a scheduling concern?
- Does business logic own time?
- Should the runtime schedule this instead?

Business capabilities should request scheduling.

They should never implement it.

Before Adding Retry Logic

Retry logic belongs to the runtime.

Contributors SHOULD NOT implement:

```
[go]
for {

    err := process()

    if err == nil {
        break
    }

    time.Sleep(... )
}
```

Instead:

Return the failure.

Allow the runtime to determine retry behaviour.

Before Creating A Worker

Ask:

- Can an existing worker pool execute this work?
- Does this require a new execution model?
- Who owns this worker?
- How does it stop?
- How is it observed?

Workers should never exist without explicit ownership.

Before Changing Runtime Behaviour

The following changes **SHOULD** require architectural review.

- delivery guarantees
- lifecycle semantics
- retry strategy
- scheduling model
- shutdown behaviour
- event contracts
- runtime APIs

These changes affect every capability.

They should never be introduced casually.

Before Merging

Every runtime contribution **SHOULD** satisfy the following checklist.

Runtime Behaviour

- Event flow remains deterministic.
- Runtime remains business agnostic.
- Worker ownership remains clear.
- Scheduling remains centralised.

Events

- Events describe facts.
- Event ownership is correct.

- Event naming follows MEG-002.
- Payloads remain immutable.

Reliability

- Subscribers remain idempotent.
- Retries remain runtime managed.
- Shutdown remains graceful.
- Failure isolation remains intact.

Observability

- Logs remain structured.
- Metrics updated where appropriate.
- Correlation preserved.
- Tracing unaffected.

Documentation

- MEG updated if required.
- ADR created where appropriate.
- Event documentation updated.
- Examples remain correct.

Architecture documentation should evolve alongside runtime behaviour.

Extension Compatibility

Contributors should assume:

Third-party extensions already exist.

Runtime changes should minimise:

- breaking changes
- behavioural surprises
- migration effort

The runtime is a platform contract.

Not merely an internal implementation.

Runtime APIs

Public runtime APIs should remain:

- minimal
- explicit
- stable
- discoverable

Every exported runtime API becomes part of the long-term platform surface.

Expanding APIs is easy.

Removing them is not.

Runtime Simplicity

When multiple implementations are possible, contributors SHOULD prefer the implementation that:

- introduces fewer concepts
- exposes fewer APIs
- requires less documentation
- remains easier to explain

Complexity should always justify itself.

Review Mindset

Runtime reviews should focus upon:

- determinism
- compatibility
- ownership
- resilience
- observability
- architectural consistency

Performance should never come at the expense of runtime correctness.

Runtime Testing

Contributors SHOULD validate:

- duplicate delivery
- retries

- shutdown
- replay
- worker cancellation
- scheduling
- event ordering
- extension compatibility

The runtime should be tested under failure as thoroughly as under success.

Learning The Runtime

New contributors SHOULD study MEG-002 in the following order.

Runtime Philosophy

↓

Event Model

↓

Event Bus

↓

Publishers

↓

Subscribers

↓

Workers

↓

Scheduling

↓

Idempotency

↓

Retries

↓

Shutdown

↓

Observability

Understanding the philosophy first makes implementation significantly easier to reason about.

Engineering Culture

Runtime contributors should strive to:

- simplify existing behaviour
- improve documentation
- reduce coupling
- preserve compatibility
- question unnecessary complexity
- document architectural reasoning

The runtime should become simpler over time.

Not more complicated.

Contributor Checklist

Before requesting review, confirm:

- ☐ The runtime remains business agnostic.
- ☐ Event contracts remain stable.
- ☐ Subscribers remain idempotent.
- ☐ Retry behaviour remains runtime owned.
- ☐ Scheduling remains centralised.
- ☐ Worker ownership is explicit.
- ☐ Shutdown remains graceful.
- ☐ Observability remains complete.
- ☐ Documentation has been updated.
- ☐ The runtime is simpler or clearer than before.

Relationship to MEG-002

This document explains how contributors should apply the architectural principles established throughout the Event-Driven Runtime specification.

The previous chapters define:

How the runtime behaves.

This chapter defines:

How engineers should evolve it.

Protecting architectural consistency is a shared responsibility.

Summary

The Event-Driven Runtime is one of the most stable components within the Mosaic platform.

Every capability depends upon it.

Every extension trusts it.

Every contributor therefore shares responsibility for preserving its:

- simplicity
- predictability
- resilience
- compatibility
- observability

The best runtime contribution is often not the one that adds the most functionality.

It is the one that allows the platform to continue evolving without increasing architectural complexity.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

18-adrs.md

Next File

glossary.md

Glossary

The runtime should speak one language. Every architectural term should have one meaning.

Purpose

This glossary defines the terminology used throughout the Mosaic Event-Driven Runtime.

Runtime terminology should remain consistent across:

- Architecture Specifications
- ADRs
- Source Code
- Documentation
- Extension SDKs
- Operational Guides

Where a term has a specific meaning within the Mosaic Runtime, that definition takes precedence over informal usage.

A

Acknowledgement (ACK)

Confirmation that a subscriber has successfully processed an event.

Acknowledgement informs the runtime that:

- the event does not require retry
- processing completed successfully

Acknowledgement does **not** imply that other subscribers have also completed successfully.

At-Least-Once Delivery

The runtime guarantee that every published event will be delivered one or more times.

Duplicate delivery is therefore expected.

Subscribers must be idempotent.

B

Backpressure

The runtime mechanism that limits incoming work when processing capacity becomes constrained.

Backpressure protects:

- memory
- CPU
- worker pools
- external systems

It exists to preserve runtime stability.

Business Event

An immutable fact describing a completed business state transition.

Examples include:

`MediaImported`

`PlaybackCompleted`

Business events belong to capabilities.

Not the runtime.

C

Capability

A self-contained unit of business functionality.

Examples include:

- Library
- Metadata
- Playback
- Search

Capabilities:

- own business state
- publish events
- subscribe to events

Capabilities do not own runtime infrastructure.

Causation ID

The identifier of the event that directly caused another event.

Causation expresses immediate parent-child relationships.

Unlike Correlation IDs, Causation IDs change throughout a workflow.

Correlation ID

An identifier shared by all events participating in the same business workflow.

Correlation allows complete workflows to be reconstructed across many capabilities.

D

Dead Letter Queue (DLQ)

The destination for events that cannot be processed successfully after retry exhaustion.

Dead-letter events require investigation.

They should never disappear silently.

Delivery

The runtime process of transporting an event from the Event Bus to a subscriber.

Delivery is owned entirely by the runtime.

E

Event

An immutable record describing a completed business fact.

Every event consists of:

- Runtime Envelope
- Business Payload

Events never describe intentions.

Only completed reality.

Event Bus

The runtime component responsible for routing events between publishers and subscribers.

The Event Bus owns:

- routing
- delivery
- retries
- acknowledgements

It intentionally owns no business logic.

Event Envelope

Runtime metadata surrounding every event.

Examples include:

- Event ID
- Version
- Correlation ID
- Producer

The envelope belongs to the runtime.

Event Payload

The immutable business information carried by an event.

Payloads describe business facts.

They never describe runtime behaviour.

I

Idempotency

The property whereby processing the same event multiple times produces the same final business state.

Idempotency is mandatory for every subscriber.

O

Occurred At

The timestamp recording when the business fact became true.

It represents business chronology.

Not delivery chronology.

P

Producer

The capability that published an event.

Every event has exactly one producer.

Publisher

The runtime component responsible for introducing events into the Event Bus.

Publishers announce facts.

They never coordinate workflows.

Q

Queue

A bounded runtime structure holding work awaiting execution.

Every runtime queue has finite capacity.

Unlimited queues are prohibited.

R

Reactive Runtime

The runtime architecture through which autonomous capabilities cooperate by publishing and subscribing to immutable events.

The Reactive Runtime owns:

- coordination
- scheduling
- retries
- workers
- observability

Business capabilities own behaviour.

Replay

The process of delivering historical events through the runtime again.

Replay enables:

- rebuilding projections
- diagnostics
- extension installation
- recovery

Replay should produce identical business behaviour to live execution.

Retry

A subsequent attempt to process a previously failed event.

Retries are owned by the runtime.

Business capabilities simply return failures.

Runtime Event

An event describing platform behaviour rather than business behaviour.

Examples include:

`WorkerStarted`

`RetryScheduled`

`BackpressureApplied`

Runtime events belong to infrastructure.

S

Scheduler

The runtime component responsible for delayed and recurring execution.

Schedulers own:

- timers
- recurring jobs
- delayed retries

They never own business decisions.

Subscriber

A capability that reacts to published events.

Subscribers:

- validate
- execute business behaviour
- publish further events

Subscribers remain autonomous.

T

Task

A unit of executable work performed by a worker.

Tasks originate from:

- events
- schedules
- retries
- runtime operations

Tasks are execution concepts.

Not business concepts.

Trace

A complete execution path describing one workflow across the runtime.

Traces combine:

- Correlation IDs
- Causation IDs
- Timing
- Capability transitions

W

Worker

A runtime component responsible for executing tasks.

Workers own execution.

They do not own business behaviour.

Worker Pool

A bounded collection of workers processing queued tasks.

Worker pools provide:

- controlled concurrency
- predictable resource usage
- operational visibility

Common Acronyms

Acronym	Meaning
ACK	Acknowledgement
ADR	Architectural Decision Record
DLQ	Dead Letter Queue
EDA	Event-Driven Architecture
ID	Identifier
MEG	Mosaic Engineering Guidelines
SDK	Software Development Kit
SWR	Stale-While-Revalidate

Relationship to MEG-002

This glossary supports every document within the Event-Driven Runtime specification.

Definitions should remain consistent across:

- runtime implementation
- extension SDKs
- architecture specifications
- operational tooling
- contributor documentation

Where runtime terminology evolves, this glossary **SHOULD** be updated before introducing new terminology elsewhere.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

19-contributor-guidance.md

Next File

references.md

References

The Mosaic Runtime is built upon established engineering principles, adapted to the unique requirements of an extensible, event-driven media platform.

Purpose

This document records the primary references that influenced the design of the Mosaic Event-Driven Runtime.

MEG-002 intentionally combines ideas from:

- Event-Driven Architecture
- Distributed Systems
- Reactive Systems
- Go Runtime Design
- Domain-Driven Design
- Cloud Native Architecture

into a cohesive runtime model designed specifically for Mosaic.

The purpose of these references is not to prescribe implementation, but to provide architectural context for the standards defined throughout MEG-002.

Event-Driven Architecture

Martin Fowler — Event-Driven Architecture

Purpose

Introduces the concepts of:

- Event Notification
- Event-Carried State Transfer
- Event Sourcing
- CQRS

Mosaic adopts the philosophy of autonomous components communicating through immutable facts while deliberately avoiding unnecessary complexity such as mandatory event sourcing.

URL

<https://martinfowler.com/articles/201701-event-driven.html>

Enterprise Integration Patterns

Authors

Gregor Hohpe

Bobby Woolf

Topics

- Message Channels
- Message Routing
- Dead Letter Queues
- Correlation
- Aggregation
- Retry
- Message Transformation

Many runtime concepts used within MEG-002 originate from these messaging patterns.

Reactive Systems

The Reactive Manifesto

Purpose

Defines the characteristics of reactive systems.

- Responsive
- Resilient
- Elastic
- Message Driven

Although Mosaic is not a Reactive Streams implementation, these principles strongly influenced the runtime architecture.

URL

<https://www.reactivemanifesto.org/>

Distributed Systems

Designing Data-Intensive Applications

Author

Martin Kleppmann

Topics

- Distributed coordination
- Event logs
- Consistency

- Replication
- Ordering
- Fault tolerance

This work heavily influenced:

- event versioning
- retries
- idempotency
- ordering philosophy
- replay

Release It!

Author

Michael T. Nygard

Topics

- Circuit Breakers
- Stability Patterns
- Bulkheads
- Operational resilience

Many runtime resilience strategies draw inspiration from this work.

Go References

Effective Go

The Go Team

Primary reference for:

- concurrency
- interfaces
- composition
- package design

https://go.dev/doc/effective_go

Go Concurrency Patterns

The Go Team

Recommended reading for:

- pipelines
- cancellation
- worker coordination

<https://go.dev/blog/pipelines>

Context

The Go Team

Reference for:

- cancellation
- deadlines
- request lifecycles

<https://go.dev/blog/context>

Share Memory By Communicating

The Go Team

Reference for Go's concurrency philosophy.

<https://go.dev/blog/share-memory-by-communicating>

Messaging Patterns

Idempotent Consumer

Chris Richardson

Reference describing safe event consumption under at-least-once delivery.

<https://microservices.io/post/microservices/patterns/2020/10/16/idempotent-consumer.html>

Transactional Outbox

Chris Richardson

Reference describing durable event publication after successful state transitions.

<https://microservices.io/patterns/data/transactional-outbox.html>

Observability

OpenTelemetry

Reference implementation for:

- traces
- metrics
- logs
- correlation

<https://opentelemetry.io/>

Prometheus

Recommended operational metrics model.

<https://prometheus.io/>

Cloud Native References

Twelve-Factor App

Several operational principles remain applicable.

Examples include:

- configuration
- disposability
- logging
- processes

Not every principle is adopted directly.

However, many complement the runtime architecture.

<https://12factor.net/>

Internal Mosaic Specifications

The following specifications complement MEG-002.

Foundation

- MEG-001 Go Engineering Standards

Planned Engineering Specifications

- MEG-003 Domain-Driven Design
- MEG-004 Hexagonal Architecture
- MEG-005 Extension Platform
- MEG-006 Runtime Architecture

- MEG-007 Storage Architecture
- MEG-008 Observability
- MEG-009 Security
- MEG-010 Performance Engineering

Mosaic Design Language

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model

Mosaic Design Specifications

- MDS-001 Design Token Architecture
- MDS-002 Colour System
- MDS-003 Material System
- MDS-004 Typography System
- MDS-005 Motion System
- MDS-006 Composition Engine
- MDS-007 Tile Framework
- MDS-008 Component Library

Runtime Principles

The runtime philosophy established throughout MEG-002 is intentionally built upon several enduring architectural principles.

These include:

- Components own behaviour.
- Events describe facts.
- Runtime owns coordination.
- Time belongs to infrastructure.
- Failures are expected.
- Delivery is independent.
- Capabilities remain autonomous.
- Observability is built-in.

- Graceful degradation is preferred over catastrophic failure.
- Simplicity scales better than cleverness.

These principles should be considered more important than any individual implementation technique.

Keeping References Current

Distributed systems continue to evolve.

Observability continues to evolve.

Go continues to evolve.

This reference list SHOULD therefore be reviewed periodically to ensure that:

- links remain valid
- recommendations remain relevant
- obsolete practices are removed
- better architectural references are incorporated

The architectural philosophy of the runtime should remain stable, even as implementation techniques improve.

Closing Statement

MEG-002 intentionally does not attempt to invent a new event-driven architecture.

Instead, it combines proven ideas from distributed systems, reactive architectures and Go engineering into a runtime specifically designed for the Mosaic platform.

The resulting architecture emphasises:

- autonomy
- simplicity
- resilience
- observability
- extensibility

Every runtime capability should reinforce these properties.

As the Mosaic ecosystem grows, the runtime should continue to feel predictable, understandable and remarkably boring.

That is not a criticism.

For infrastructure, boring is one of the highest compliments an engineer can give.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`glossary.md`

Next File

End of Specification